

TECNOLOGIA DA INFORMAÇÃO

Linguagem Java



Presidente: Gabriel Granjeiro

Vice-Presidente: Rodrigo Calado

Diretor Pedagógico: Erico Teixeira

Diretora de Produção Educacional: Vivian Higashi

Gerente de Produção Digital: Bárbara Guerra

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

CÓDIGO:

250605175149



PATRÍCIA QUINTÃO

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC – Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista – TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

GRAN
CONCURSOS

SUMÁRIO

Apresentação	5
Linguagem Java	6
1. Termos, Definições e Abreviaturas.....	6
2. Introdução ao Java.....	8
2.1. História.....	9
2.2. Características Principais	11
2.3. Funcionamento da JVM, Bytecode e Garbage Collector	11
2.4. Ambiente de Desenvolvimento	15
2.5. Sintaxe Básica da Linguagem.....	16
3. Fundamentos da Linguagem.....	18
3.1. Variáveis e Tipos de Dados.....	18
3.2. Declaração e Inicialização de Variáveis.....	21
3.3. Operadores	21
3.4. Entrada e Saída de Dados	24
4. Estruturas de Controle	25
4.1. Estruturas Condicionais.....	25
4.2. Estruturas de Repetição	28
4.3. Comandos de Controle de Fluxo	31
5. Funções e Modularização.....	32
5.1. Definição e Chamada de Métodos	32
5.2. Parâmetros e Retorno de Métodos.....	34
5.3. Sobrecarga de Métodos	35
5.4. Escopo de Variáveis	36
6. Estruturas de Dados Básicas.....	37
6.1. Arrays	37
6.2. Strings	40

6.3. Coleções (ArrayList)	42
7. Programação Orientada a Objetos (POO)	43
7.1. Conceitos Fundamentais	44
7.2. Encapsulamento	47
7.3. Herança	50
7.4. Polimorfismo	51
7.5. Classes e Métodos Abstratos	53
7.6. Interfaces	55
8. Tratamento de Exceções	57
8.1. Conceito	58
8.2. Blocos try-catch	59
8.3. Lançamento de Exceções	60
8.4. Exceções Comuns	61
Resumo	63
Questão Comentada em Aula	68
Exercícios	69
Gabarito	87
Gabarito Comentado	88
Referências	127

APRESENTAÇÃO

Olá, querido(a) amigo(a), tudo bem?

“O Poder do Entusiasmo”: Descubra o entusiasmo na Vida!!

Nós somos capazes de transformar as coisas e fazê-las acontecer. Faça o entusiasmo ocorrer pela certeza de que você é capaz de realizações eficazes e de... **VENCER OBSTÁCULOS E SER FELIZ!!**

Trago aqui a aula sobre **JAVA** e tópicos relacionados. Espero que aproveite!

Continue firme nessa caminhada e ótimos estudos!

Um abraço,

Profª Patrícia Lima Quintão

LINGUAGEM JAVA

1. TERMOS, DEFINIÇÕES E ABREVIATURAS

A seguir destacamos alguns termos/definições e abreviaturas:

- **abstract:** palavra-chave usada para declarar uma classe ou método abstrato. Uma classe abstrata não pode ser instanciada diretamente e pode conter métodos com ou sem implementação. Já um método abstrato não possui corpo e deve obrigatoriamente ser implementado por uma subclasse concreta;
- **array:** estrutura de dados que armazena múltiplos valores do mesmo tipo, acessados por índices. Arrays em Java são objetos e possuem um comprimento fixo, definido no momento da sua criação;
- **bytecode:** código intermediário gerado pelo compilador Java (javac) a partir do código-fonte `.java`. O bytecode é independente de plataforma e é executado pela JVM, garantindo portabilidade;
- **catch:** bloco utilizado para tratar exceções lançadas em um bloco `try`. Permite capturar tipos específicos de exceção e definir o que deve ser feito quando elas ocorrem;
- **class:** palavra-chave usada para definir uma nova classe. Uma classe é um modelo para criar objetos em Java;
- **default (interface):** desde o Java 8, permite implementar métodos com corpo dentro de interfaces;
- **enum:** tipo especial de classe que representa um conjunto fixo de constantes. Muito usado para representar estados ou opções;
- **extends:** palavra-chave usada para indicar que uma classe herda os atributos e comportamentos (métodos) de outra classe. Implementa o conceito de herança em Java;
- **garbage collector:** componente da JVM responsável por gerenciar a memória automaticamente, eliminando objetos que não são mais acessíveis e liberando espaço;
- **final:** palavra-chave usada para declarar constantes ou impedir alterações. Pode ser aplicada a: Variáveis (impede que o valor seja alterado), Métodos (impede que sejam sobrescritos), Classes (impede que sejam herdadas);
- **finally:** bloco de código que sempre é executado após um `try` ou `catch`, independentemente de ter ocorrido ou não uma exceção. É útil para liberar recursos como arquivos ou conexões;
- **implements:** palavra-chave usada por uma classe para implementar uma ou mais interfaces. Exige que todos os métodos da interface sejam definidos na classe;

- **import:** permite a inclusão de classes de outros pacotes no código, facilitando o acesso a bibliotecas e APIs;
- **interface:** define um contrato que outras classes podem implementar. Contém apenas métodos abstratos (até o Java 7) e constantes;
- **instanceof:** operador utilizado para verificar se um objeto pertence a uma classe ou interface específica. Retorna `true` ou `false`;
- **Java:** linguagem de programação orientada a objetos, de propósito geral e com foco em portabilidade, segurança e robustez. Muito utilizada em aplicações corporativas, Android, sistemas embarcados e aplicações web;
- **JVM (Java Virtual Machine):** máquina virtual que executa o bytecode Java. Garante que os programas escritos em Java possam ser executados em qualquer sistema operacional que tenha uma JVM instalada;
- **método:** função declarada dentro de uma classe que descreve uma ação que os objetos daquela classe podem realizar;
- **multithread:** suporta múltiplas tarefas;
- **native:** palavra-chave usada para declarar métodos implementados fora do Java, geralmente em C ou C++;
- **new:** palavra-chave utilizada para instanciar objetos e criar arrays. Aloca memória e invoca o construtor da classe;
- **objeto:** instância de uma classe. Ele representa uma entidade concreta que combina estado (atributos/variáveis) e comportamento (métodos) definidos pela classe;
- **package:** palavra-chave usada para organizar classes em namespaces. Facilita o agrupamento lógico de classes relacionadas;
- **POO:** programação orientada a objetos;
- **public:** modificador de acesso que permite que um método, campo ou classe seja acessado de qualquer outro lugar do programa, mesmo de pacotes diferentes;
- **private:** modificador de acesso que permite que um método ou campo seja acessado apenas dentro da própria classe em que foi declarado;
- **protected:** modificador de acesso que permite o acesso dentro do mesmo pacote e também em subclasses, mesmo que estejam em pacotes diferentes;
- **return:** palavra-chave que finaliza a execução de um método e pode retornar um valor;
- **static:** palavra-chave que define que um atributo ou método pertence à classe, e não a uma instância específica. Pode ser acessado sem criar um objeto da classe;
- **string:** em Java, string é uma classe utilizada para representar uma sequência de caracteres. Objetos do tipo string são imutáveis, ou seja, uma vez criados, seu conteúdo não pode ser alterado;

- **tipo primitivo:** tipos de dados básicos e nativos da linguagem Java, como ``int'`, ``float'`, ``boolean'`, ``char'`, ``byte'`, entre outros. Armazenam diretamente valores simples e não possuem métodos;
- **tipo de referência:** representa objetos, arrays, interfaces e classes. Armazenam endereços de memória e, por padrão, são inicializados com ``null'`;
- **this:** palavra-chave usada dentro de uma classe para referenciar o próprio objeto. É útil para distinguir entre variáveis de instância e parâmetros com o mesmo nome;
- **throw:** palavra-chave usada para lançar uma exceção manualmente;
- **throws:** palavra-chave usada para declarar que um método pode lançar exceções;
- **try:** bloco de código que envolve instruções que podem lançar exceções. Deve ser acompanhado de pelo menos um bloco ``catch'` ou ``finally'`;
- **visibilidade default (package-private):** quando nenhum modificador de acesso é especificado, o membro é acessível apenas dentro do mesmo pacote. Essa visibilidade é chamada de “default” ou “package-private”;
- **void:** palavra-chave que indica que um método não retorna valor. É utilizada na declaração de métodos que realizam uma ação sem retorno;
- **WORA:** white once, run anywhere (escreva uma vez, execute em qualquer lugar).

2. INTRODUÇÃO AO JAVA

Desenvolvida com foco em portabilidade, robustez e segurança, Java rapidamente se tornou uma das linguagens mais utilizadas do mundo, especialmente em aplicações corporativas, dispositivos móveis (Android) e sistemas embarcados.

Também é conhecida por sua capacidade **de escrever uma vez, executar em qualquer lugar**, o que significa que o código escrito em Java pode ser executado em qualquer dispositivo que suporte a **Java Virtual Machine (JVM)**.

A sintaxe e estrutura Java são semelhantes às linguagens baseadas em C, como o C. Sua robustez, compatibilidade independente de plataforma e gerenciamento de memória forte tornaram uma linguagem para desenvolvedores em todo o mundo.

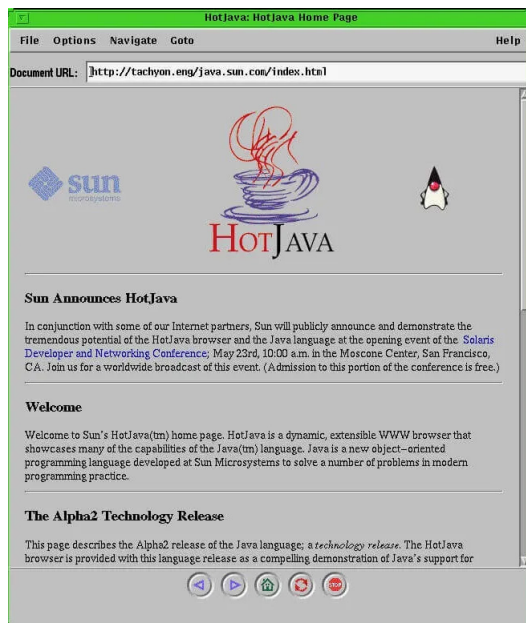


2.1. HISTÓRIA

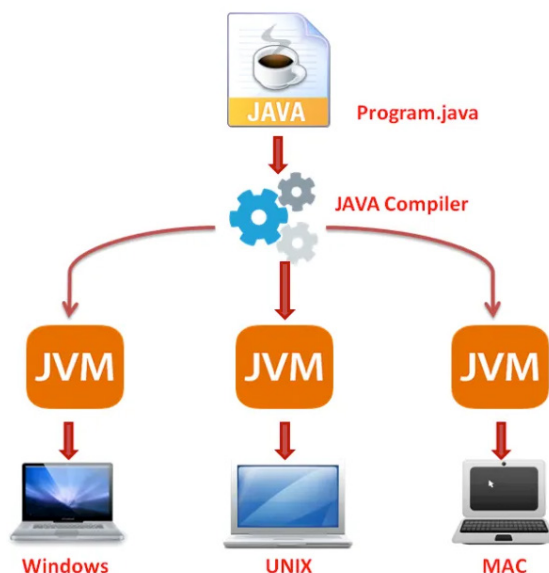
Java foi desenvolvida pela **Sun Microsystems** (adquirida pela Oracle em 2010) no início da década de 1990, mais precisamente em 1991, por uma equipe liderada por James Gosling. Inicialmente chamada de **Oak**, a linguagem foi concebida para ser utilizada em dispositivos eletrônicos inteligentes e eletrodomésticos. No entanto, com o advento da internet e a necessidade de uma linguagem que pudesse rodar em diferentes sistemas operacionais (Windows, macOS, Linux, etc.), nasceu o **projeto WebRunner**, um navegador que tinha a proposta de trazer interatividade na web. Foi lançada oficialmente como **Java** em 1995.

Foi nessa época que o Java se tornou popular. Com o WebRunner, que foi renomeado para **HotJava**, era possível exibir conteúdo HTML (algo que os navegadores da época já faziam), mas também criar páginas interativas com Applets.

Foi nessa época que o Java se tornou popular. Com o WebRunner, que foi renomeado para HotJava, era possível exibir conteúdo HTML (algo que os navegadores da época já faziam), mas também criar páginas interativas com Applets.



Além disso, o Java foi a primeira linguagem a oferecer portabilidade real entre plataformas, um diferencial essencial na era da internet, em que sistemas precisavam ser acessados por diferentes tipos de computadores. Essa característica deu origem ao famoso slogan da linguagem: **WORA — Write Once, Run Anywhere** (escreva uma vez, execute em qualquer lugar).



Após o sucesso inicial da linguagem, em 1996 foi lançada uma versão aprimorada com diversas atualizações, batizada de JDK (Java Development Kit) — um kit completo para o desenvolvimento de aplicações em Java. A partir daí, a linguagem se expandiu rapidamente, dando origem a diferentes edições voltadas para públicos e finalidades específicas: Java SE

(Standard Edition), Java EE (Enterprise Edition), Java ME (Micro Edition) para dispositivos móveis, Java Card para smartcards e Java TV para aplicações em televisores. Em 2008, a plataforma foi adquirida pela Oracle, que até hoje é responsável pela sua manutenção, oferecendo diferentes modelos de licenciamento para uso pessoal e comercial.

2.2. CARACTERÍSTICAS PRINCIPAIS

O sucesso e a longevidade do Java se devem a várias características marcantes:

- **Orientação a Objetos:** Java é uma linguagem puramente **orientada a objetos (POO)**. Isso significa que tudo nela é tratado como um objeto, facilitando a organização, reutilização e manutenção do código;
- **Portabilidade: Write Once, Run Anywhere (WORA) – Escreva uma vez, execute em qualquer lugar.** Esta é a característica mais icônica do Java. Um programa Java compilado pode ser executado em qualquer dispositivo que tenha uma JVM instalada, independentemente do sistema operacional ou da arquitetura de hardware;
- **Simplicidade:** comparada a linguagens como C++, Java foi projetada para ser mais simples de aprender e usar, eliminando ponteiros e gerenciamento manual de memória, o que reduz erros comuns;
- **Segurança:** possui um forte modelo de proteção que combina diversos recursos. A linguagem evita ponteiros explícitos, realiza verificação de tipos em tempo de compilação e oferece uma sandbox para execução de código não confiável. Java também conta com diversas ferramentas e APIs voltadas à segurança;
- **Multithreaded:** suporta a execução de múltiplas tarefas (threads) simultaneamente dentro de um mesmo programa, o que é essencial para aplicações modernas e responsivas;
- **Robustez:** possui mecanismos robustos para tratamento de erros (exceções) e gerenciamento automático de memória (Garbage Collection), minimizando falhas e tornando as aplicações mais estáveis;
- **De Alto Desempenho:** apesar de ser interpretada pela JVM, Java utiliza tecnologias de otimização como o compilador Just-In-Time (JIT) que a tornam muito eficiente, aproximando o desempenho de linguagens compiladas.

2.3. FUNCIONAMENTO DA JVM, BYTECODE E GARBAGE COLLECTOR

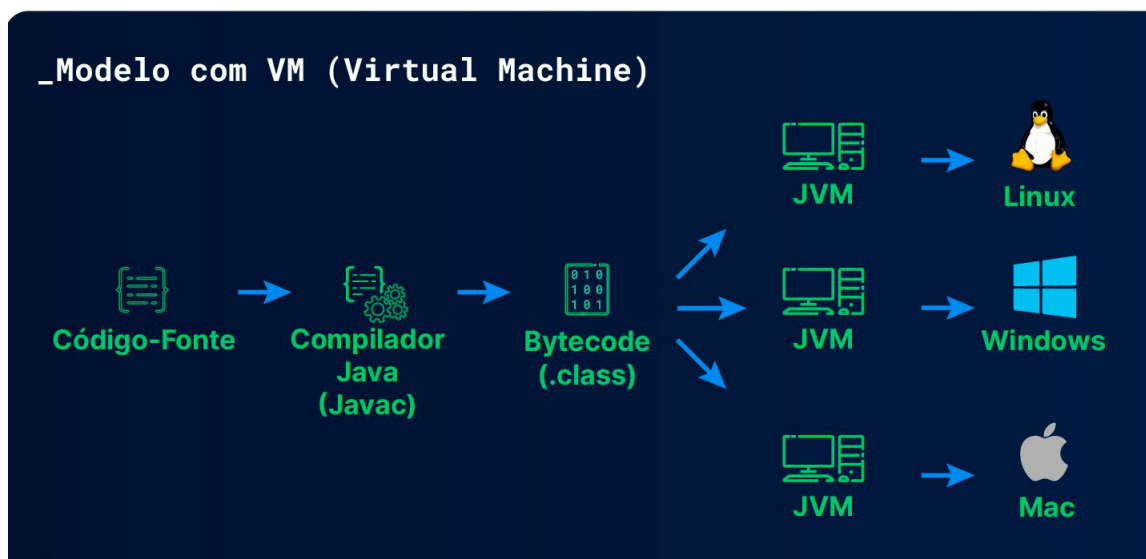
Para compreender a arquitetura de execução do Java, é essencial analisar o funcionamento da JVM (Máquina Virtual Java), o papel do bytecode na portabilidade entre plataformas e a importância do Garbage Collector na gestão automática de memória dentro do ecossistema Java.

2.3.1. JVM (JAVA VIRTUAL MACHINE)

A **Máquina Virtual Java (JVM)** é o coração da plataforma Java e a responsável pela sua característica de **portabilidade**. Consiste em uma máquina virtual, um computador abstrato que tem seu próprio ISA, memória, pilha etc. Ele é executado no sistema operacional host e coloca suas demandas por recursos nele.

A principal função da JVM é **interpretar e executar o bytecode Java**. Quando você escreve um programa em Java, ele não é compilado diretamente para o código de máquina do seu computador. Em vez disso, ele é compilado para um formato intermediário chamado **bytecode**. A JVM atua como uma camada de abstração entre o bytecode e o hardware subjacente.

Imagine a JVM como um “computador virtual” que entende a linguagem bytecode. Cada sistema operacional (Windows, Linux, macOS) tem sua própria versão da JVM, mas todas elas são capazes de executar o mesmo bytecode Java.



Como a JVM executa o código Java? O motor de execução compreende o coletor de lixo e o compilador JIT. A JVM vem de duas formas – **cliente** e **servidor**. Ambos compartilham o mesmo código de tempo de execução, mas diferem no que o JIT é usado. O usuário pode controlar qual modo usar especificando as bandeiras JVM -client ou -server. O servidor JVM foi projetado para aplicativos Java de longa duração em servidores.

O processo de execução envolve as seguintes etapas:

1. Carregamento de Classes (Class Loader): a JVM carrega os arquivos.class (que contêm o bytecode) na memória.

2. Verificação de Bytecode (Bytecode Verifier): antes da execução, a JVM verifica a segurança e a correção do bytecode, garantindo que ele não viole as regras da linguagem ou comprometa o sistema.

3. Execução (Execution Engine):

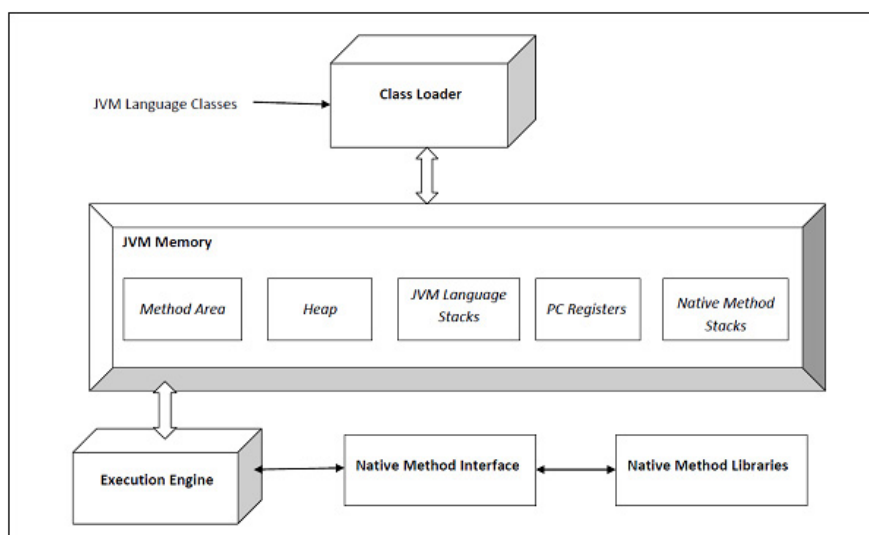
- **Interpretador:** a JVM pode interpretar o bytecode linha por linha, convertendo-o em código de máquina e executando-o;
- **Compilador Just-In-Time (JIT):** para otimizar o desempenho, a JVM utiliza um compilador JIT. Ele identifica partes do código que são executadas com frequência (os “hotspots”) e as compila em código de máquina nativo durante a execução. Isso torna as execuções subsequentes daquele trecho de código muito mais rápidas.

4. Gerenciamento de memória (Garbage Collector): a JVM gerencia a memória automaticamente através do Garbage Collector. Ele identifica e libera a memória que não está mais sendo utilizada pelos objetos, evitando vazamentos de memória e simplificando o trabalho do desenvolvedor.

5. Encerramento: quando o programa termina, a JVM limpa todos os recursos e se encerra de maneira ordenada.

Em resumo: o código-fonte Java (.java) é compilado para bytecode (.class). A JVM carrega, verifica e otimiza esse bytecode, traduzindo-o para instruções que o sistema operacional e o hardware podem entender, permitindo a execução do programa em qualquer lugar.

Arquitetura do HotSpot JVM 3

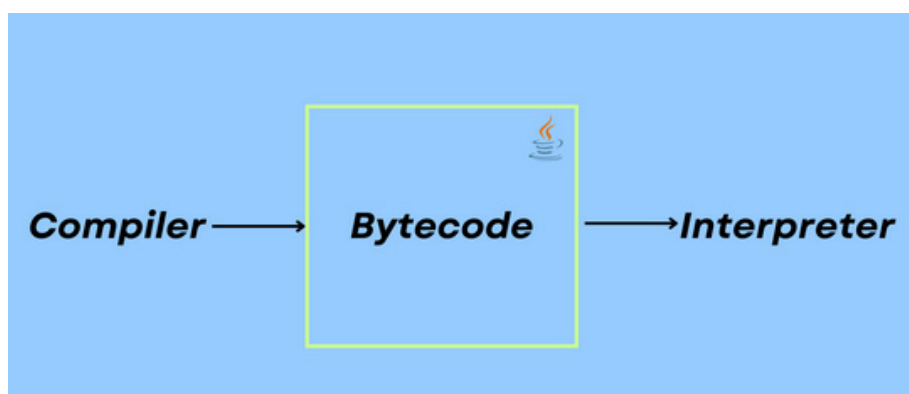


2.3.2. BYTECODE

O bytecode é uma representação intermediária do código Java, gerada após a compilação de um arquivo.java. Ao invés de ser convertido diretamente para código de máquina específico de uma plataforma, como ocorre em linguagens compiladas tradicionais, o compilador Java (javac) transforma o código fonte em bytecode, que é armazenado em arquivos com a extensão.class.

Esse bytecode não é executado diretamente pelo sistema operacional. Em vez disso, ele é interpretado ou compilado em tempo de execução pela **JVM (Java Virtual Machine)**, que atua como uma camada intermediária entre o programa e o sistema operacional. Isso é o que garante uma das principais vantagens do Java: a **portabilidade**. Um mesmo arquivo.class pode ser executado em qualquer dispositivo que tenha uma JVM compatível, independentemente do sistema operacional ou da arquitetura do hardware. Esse conceito é conhecido como **WORA** — *Write Once, Run Anywhere* (escreva uma vez, execute em qualquer lugar).

Além disso, o bytecode facilita a aplicação de técnicas de segurança, otimização e análise de código em tempo de execução, já que seu formato padronizado e intermediário permite que a JVM inspecione e gerencie a execução de forma eficiente e controlada.



2.3.3. GARBAGE COLLECTOR

O **Garbage Collector (ou coletor de lixo)** é um componente fundamental da JVM, responsável por realizar a **gerência automática de memória** em aplicações Java. Sua principal função é identificar e remover da memória objetos que não são mais utilizados pelo programa, liberando espaço e evitando vazamentos de memória.

Em Java, quando um objeto é criado com a palavra-chave `new`, ele é alocado na memória heap. À medida que o programa é executado, muitos desses objetos deixam de ser necessários, mas o desenvolvedor não precisa se preocupar em desalocá-los manualmente — essa tarefa é feita automaticamente pelo Garbage Collector.

Esse processo torna o desenvolvimento em Java mais seguro e produtivo, já que reduz a chance de erros como *dangling pointers* (ponteiros soltos) e *memory leaks* (vazamentos de memória), comuns em linguagens que exigem gerenciamento manual, como C e C++.

O Garbage Collector atua periodicamente, analisando quais objetos ainda estão acessíveis (ou “alcançáveis”) por referências ativas no código. Objetos inacessíveis são considerados lixo e podem ser removidos.

A JVM implementa diferentes **algoritmos de coleta**, como:

- **Mark and Sweep** (marcar e varrer);
- **Generational Collection** (coleta geracional): a coleta geracional melhora o desempenho, pois permite que o coletor de lixo se concentre mais nas áreas da memória em que a coleta tem maior impacto;
- **G1 (Garbage-First)** — que é eficiente para aplicações de grande porte.



2.4. AMBIENTE DE DESENVOLVIMENTO

- **JDK (Java Development Kit):** o **JDK** é o conjunto completo de ferramentas de desenvolvimento Java. Ele inclui:
 - **JRE (Java Runtime Environment):** o ambiente necessário para *executar* programas Java;
 - **Ferramentas de Desenvolvimento:** o compilador (javac), o launcher da JVM (java), o depurador (jdb) etc.;
 - **Bibliotecas (APIs):** um vasto conjunto de classes e interfaces prontas para uso que facilitam o desenvolvimento (ex.: para manipular strings, coleções de dados, redes etc.);
- **IDEs (Integrated Development Environments):** embora se possa programar Java usando um editor de texto simples e o terminal, as IDEs tornam o desenvolvimento muito mais produtivo. Elas oferecem recursos como:
 - Editor de código com realce de sintaxe;
 - Autocompletar código;
 - Ferramentas de depuração (para encontrar e corrigir erros);
 - Integração com compiladores e ferramentas de build;
 - Navegação de código.

As IDEs mais populares para Java incluem:

- **IntelliJ IDEA:** muito completa e robusta (versões Community gratuita e Ultimate paga);
- **Eclipse:** uma IDE open-source amplamente utilizada, especialmente em ambientes corporativos;
- **NetBeans:** outra IDE open-source popular e fácil de usar;
- **VS Code (com extensões Java):** leve e versátil, mas precisa de extensões para ter o suporte completo a Java.

2.5. SINTAXE BÁSICA DA LINGUAGEM

A sintaxe de uma linguagem de programação são as regras que definem como você deve escrever o código para que o compilador possa entendê-lo. A sintaxe de Java é baseada em C e C++, clara, fortemente tipada, mas com algumas simplificações.

Quando consideramos um programa Java, ele pode ser definido como uma coleção de objetos que se comunicam através da invocação dos métodos uns dos outros. Abaixo estão os principais elementos:

- **Classes e objetos:** toda aplicação Java é baseada em classe. O código é organizado dentro de classes e os objetos são instâncias dessas classes;

```
class Pessoa {
    String nome;
}
```

- **Métodos:** blocos de código que definem comportamentos. São declarados dentro de classes;

```
void falar() {
    System.out.println("Olá!");
}
```

- **Tipos de dados:** primitivos (int, float, char, boolean etc.) e de Referência (string, arrays, objetos personalizados);

- **Variáveis:** armazenam valores e podem ser locais, de instância ou estáticas;

```
int idade = 30;
```

- **Operadores:** matemáticos (+, -, *, /), relacionais (==, !=, >, <), lógicos (&&, ||, !);

- **Estruturas de controle:** condicionais (if, else, switch) e laços (for, while, do-while);

- **Modificadores de acesso:** controlam a visibilidade de classes, métodos e atributos (public, private, protected, default);

- **Palavra-chave main:** método de entrada do programa Java;

```
public static void main(String[] args) {
    // Código a ser executado
}
```

- **Uso de new:** cria novos objetos;

```
Pessoa p = new Pessoa();
```

- **Sintaxe de blocos:** blocos de código são delimitados por chaves {}. Cada instrução termina com ponto e vírgula;

Pontos importantes sobre os programas Java:

- **Case Sensitivity** – Java é case sensitivity, o que significa que o identificador **Hello** e **olá** teria um significado diferente em Java;
- **Nomes de classe** – Para todos os nomes de classe, a primeira letra deve estar em Upper Case. Se várias palavras são usadas para formar um nome da classe, a primeira letra de cada palavra interior deve estar em Upper Case;

EXEMPLO

```
class MyFirstJavaClass
```

- **Nomes de método** – Todos os nomes dos métodos devem começar com uma letra de caso inferior. Se várias palavras são usadas para formar o nome do método, então a primeira letra de cada palavra interna deve estar em Upper Case;

EXEMPLO

```
public void myMethodName()
```

- **Nome do arquivo do programa** – Nome do arquivo do programa deve corresponder exatamente ao nome da classe. Ao salvar o arquivo, você deve salvá-lo usando o nome da classe (Remember Java é maiúsculo) e anexar 'java' ao final do nome (se o nome do arquivo e o nome da classe não corresponderem, seu programa não compilará). Entretanto, caso você não tenha uma classe pública presente no arquivo, o nome do arquivo pode ser diferente do nome da classe. Também não é obrigatório ter uma classe pública no arquivo;

EXEMPLO

Assuma 'MyFirstJavaProgram' é o nome da classe. Em seguida, o arquivo deve ser salvo como 'MyFirstJavaProgram.java'

- **public static void main(String args[])** – O processamento de programas Java começa a partir do método main(), que é uma parte obrigatória de cada programa Java.

3. FUNDAMENTOS DA LINGUAGEM

Este capítulo apresenta os conceitos essenciais da linguagem Java, como **variáveis**, **tipos de dados**, **operadores** e **mecanismos de entrada e saída de informações**, que constituem a base para o desenvolvimento de qualquer programa.

3.1. VARIÁVEIS E TIPOS DE DADOS

Em Java, todo dado que manipulamos em nossos programas deve ter um tipo. Essa tipagem ajuda o compilador a entender como os dados devem ser armazenados na memória e quais operações podem ser realizadas sobre eles. Os tipos de dados são classificados em duas categorias principais: **Tipos Primitivos** e **Tipos de Referência**.

3.1.1. TIPOS PRIMITIVOS

Os **tipos primitivos** são os blocos de construção fundamentais da linguagem Java. Eles representam valores simples e diretos, sem a complexidade de objetos.

Características principais:

- **Armazenam valores diretos:** uma variável de tipo primitivo armazena o valor real diretamente na sua localização de memória;
- **Tamanho fixo e pré-definido:** cada tipo primitivo possui um tamanho fixo em bits e um intervalo de valores que pode representar, definidos pela especificação Java. Isso garante que o comportamento do seu código seja consistente em diferentes plataformas;
- **Não podem ser null:** uma variável de tipo primitivo sempre contém um valor; ela não pode estar “vazia” ou sem um valor atribuído (null);
- **Alocação na Stack (Pilha):** geralmente, variáveis de tipos primitivos são alocadas na **stack** (pilha de execução), que é uma área de memória mais rápida e organizada para dados temporários de métodos;
- **Valor padrão:** possuem um valor padrão (geralmente zero, falso ou ‘\u0000’) se não forem explicitamente inicializados.

Os 8 Tipos Primitivos de Java:

- **Numéricos Inteiros:**
 - byte (8 bits)
 - short (16 bits)
 - int (32 bits) – Mais comum para inteiros. Valor padrão: 0.
 - long (64 bits)
- **Numéricos de Ponto Flutuante (Decimais):**
 - float (32 bits)
 - double (64 bits) – Mais comum para decimais. Valor padrão: 0.0.

- **Caractere:**
 - char (16 bits) – Representa um único caractere Unicode. Valor padrão: \u0000 (caractere nulo).
- **Booleano:**
 - boolean (tamanho não estritamente definido, mas opera como 1 bit para true/false) – Representa valores lógicos true ou false. Valor padrão: false.

EXEMPLO

```
public class Variaveis {
    public static void main(String[] args) {
        int idade = 25;
        double altura = 1.75;
        char inicial = 'J';
        boolean ativo = true;
        System.out.println("Idade: " + idade + ", Altura: " + altura + ",
Inicial: " + inicial + ", Ativo: " + ativo);
    }
}
```

Saída: Idade: 25, Altura: 1.75, Inicial: J, Ativo: true

Tipos primitivos de Java e suas principais características

Tipo	Tamanho (bits)	Valor Padrão	Intervalo de Valores	Exemplo de Uso
byte	8	0	-128 a 127	byte idade = 25;
short	16	0	-32.768 a 32.767	short ano = 2025;
int	32	0	-2^{31} a $2^{31} - 1$ (aprox. -2,1 bi a 2,1 bi)	int total = 1000;
long	64	0L	-2^{63} a $2^{63} - 1$	long populacao = 8_000_000_000L;
float	32	0.0f	$\pm 3.4 \times 10^{38}$ (precisão simples, ~7 dígitos)	float preco = 10.99f;
double	64	0.0d	$\pm 1.8 \times 10^{308}$ (precisão dupla, ~15 dígitos)	double pi = 3.14159;
char	16	'\u0000'	0 a 65.535 (tabela Unicode)	char letra = 'A';
boolean	1 (não definido precisamente)	false	true ou false	boolean ativo = true;

3.1.2. TIPOS DE REFERÊNCIA

Ao contrário dos tipos primitivos, os **tipos de referência** não armazenam o valor real diretamente na variável. Em vez disso, eles armazenam uma referência (um endereço de memória) para o local em que o objeto real está armazenado na memória.

Características principais:

- **Armazenam referências:** uma variável de tipo de referência “aponta” para um objeto. O objeto em si reside em outra área da memória;
- **Alocação na Heap:** os objetos, que são os “valores” dos tipos de referência, são alocados na **heap**, uma área de memória dinâmica onde os objetos vivem durante a execução do programa e são gerenciados pelo Garbage Collector;
- **Podem ser null:** uma variável de tipo de referência pode não apontar para nenhum objeto. Nesse caso, ela contém o valor especial null. Tentar acessar um método ou campo de uma referência null resultará em um NullPointerException;
- **Criados pelo programador ou fornecidos pela linguagem:** incluem classes, interfaces, e arrays, muitos dos quais são definidos pelo próprio programador, além de classes padrão da linguagem (como String).

Exemplos importantes de tipos de referência:

- **Classes:** qualquer classe que você cria (Ex.: Pessoa, Produto, Carro) é um tipo de referência. Objetos (instâncias) dessas classes serão do tipo de referência correspondente;
- **Interfaces:** definem um contrato para classes, mas são tipos de referência;
- **Arrays:** coleções de elementos (sejam primitivos ou de referência). Um array em si é um objeto;
- **String:** embora pareça um tipo primitivo pela sua facilidade de uso, String é uma classe (e, portanto, um tipo de referência) em Java, representando sequências de caracteres;
- **Enums:** tipos de enumeração;
- **Wrapper Classes:** classes que “embrulham” os tipos primitivos em objetos (Ex.: Integer para int, Double para double, Boolean para boolean).

EXEMPLO

```
String nome = "Flávia"; // 'nome' guarda a referência para o objeto String
                        "Flávia"
Scanner leitor = new Scanner(System.in); // 'leitor' guarda a referência para
um objeto Scanner
int[] numeros = {1, 2, 3}; // 'numeros' guarda a referência para um objeto
array de inteiros
```

```
Pessoa minhaPessoa = new Pessoa(); // 'minhaPessoa' guarda a referência para
um objeto Pessoa
List<String> listaNomes = new ArrayList<>(); // 'listaNomes' guarda a referência
para um objeto ArrayList
```

3.2. DECLARAÇÃO E INICIALIZAÇÃO DE VARIÁVEIS

- **Declaração:** define o tipo e o nome da variável;
- **Inicialização:** atribui um valor inicial à variável;
- **Regras:**
 - O nome da variável deve começar com letra, _ ou \$ ou \$, seguido por letras, números, _ ou \$;
 - Java é case-sensitive (ex. idade ≠ Idade);
 - Variáveis devem ser inicializadas antes de usadas, ou o compilador gerará erro.

EXEMPLO

```
public class Declaracao {
    public static void main(String[] args) {
        int numero; // Declaração
        numero = 10; // Inicialização
        double preco = 99.99; // Declaração e inicialização
        System.out.println("Número: " + numero + ", Preço: " + preco);
    }
}
```

Saída: Número: 10, Preço: 99.99

Erro comum:

```
int x;
System.out.println(x); // Erro: variável não inicializada
```

3.3. OPERADORES

Os operadores são símbolos usados para realizar operações com variáveis e valores. Eles são divididos em categorias principais.

3.3.1. ARITMÉTICOS

Usados para realizar operações matemáticas básicas:

- +: adição;
- -: subtração;
- *: multiplicação;
- /: divisão;
- %: módulo (resto da divisão).

EXEMPLO

```
public class Aritmeticos {
    public static void main(String[] args) {
        int a = 10, b = 3;
        System.out.println("Soma: " + (a + b)); // 13
        System.out.println("Divisão: " + (a / b)); // 3 (divisão inteira)
        System.out.println("Resto: " + (a % b)); // 1
    }
}
```

Obs.: Divisão por zero com inteiro causa `ArithmeticException`.

3.3.2. RELACIONAIS

Usados para comparar dois valores e retornar um valor booleano (verdadeiro ou false) com base no resultado da comparação:

- ==: igual;
- !=: diferente;
- >: maior que;
- <: menor que;
- >=: maior ou igual;
- <=: menor ou igual.

EXEMPLO

```
public class Relacionais {
    public static void main(String[] args) {
        int x = 5, y = 8;
        System.out.println("x == y: " + (x == y)); // false
        System.out.println("x < y: " + (x < y)); // true
    }
}
```

3.3.3. LÓGICOS

Usados para realizar operações lógicas (and, or ou not) em valores booleanos:

- &&: e (and);
- ||: ou (or);
- !: não (not).

EXEMPLO

```
public class Logicos {
    public static void main(String[] args) {
        boolean a = true, b = false;
        System.out.println("a && b: " + (a && b)); // false
        System.out.println("a || b: " + (a || b)); // true
        System.out.println("!a: " + (!a)); // false
    }
}
```

3.3.4. OPERADORES DE ATRIBUIÇÃO E INCREMENTO/DECREMENTO

Usados para atribuir um valor a uma variável:

- =: atribuição;
- +=, -=, *=, /=, %+=: atribuição composta;
- ++: incremento (adiciona 1);
- --: decremento (subtrai 1).

EXEMPLO

```
public class Atribuicao {
    public static void main(String[] args) {
        int x = 5;
        x += 3; // x = x + 3
        System.out.println("x após += 3: " + x); // 8
        x++; // x = x + 1
        System.out.println("x após ++: " + x); // 9
    }
}
```

Categoria	Operador(es)	Descrição	Exemplo
Aritméticos	+ - * / %	Soma, subtração, multiplicação, divisão, módulo	a + b, a % b

Categoria	Operador(es)	Descrição	Exemplo
Relacionais	== != > < >= <=	Comparação de igualdade/desigualdade e ordem	a != b, x <= y
Lógicos	&& !	Representa E, OU ou NÃO	a && b
Atribuição	= += -= *= /= %=	Atribuição simples e composta	x += 10 → x = x + 10

3.4. ENTRADA E SAÍDA DE DADOS

Saída de dados: a classe `System.out` é usada para exibir informações no console.

- **`System.out.println()`:** exibe texto e adiciona uma nova linha;
- **`System.out.print()`:** exibe texto sem nova linha;
- **`System.out.printf()`:** formata a saída (ex.: `%.2f` para duas casas decimais).

Entrada de dados: a classe **Scanner** (pacote `java.util`) é usada para ler entrada do usuário.

EXEMPLO

```
import java.util.Scanner;

public class EntradaSaida {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite seu nome: ");
        String nome = scanner.nextLine();

        System.out.print("Digite sua idade: ");
        int idade = scanner.nextInt();

        System.out.printf("Olá, %s! Você tem %d anos.%n", nome, idade);
        scanner.close(); // Fecha o Scanner
    }
}
```

Explicação:

- `nextLine()`: lê uma linha de texto;
- `nextInt()`: lê um número inteiro;
- Sempre feche o Scanner com `close()` para liberar recursos.

Pergunta: Qual método do Scanner é usado para ler uma string?

Resposta: `nextLine()`

DICA

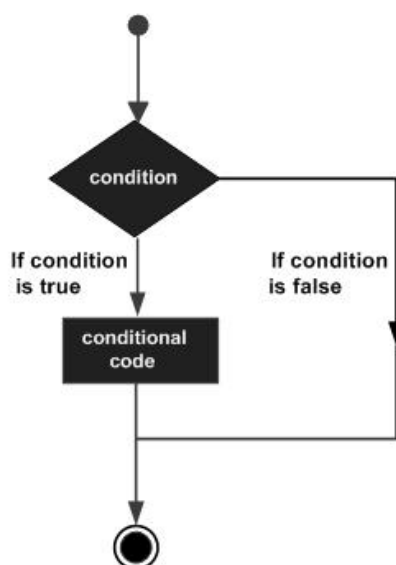
Se a pergunta exigir a leitura de múltiplos valores, use `next()` para palavras individuais e `nextLine()` para linhas completas.

4. ESTRUTURAS DE CONTROLE

As estruturas de controle em Java, que permitem direcionar o fluxo de execução de um programa com base em condições e repetições. Esses conceitos são essenciais para resolver problemas de lógica em programação.

As estruturas de tomada de decisão têm uma ou mais condições a serem avaliadas ou testadas pelo programa, juntamente com uma declaração ou instruções que devem ser executadas se a condição for determinada como verdadeira e, opcionalmente, outras declarações a serem executadas se a condição for considerada falsa.

A seguir está a forma geral de uma estrutura de tomada de decisão típica encontrada na maioria das linguagens de programação.

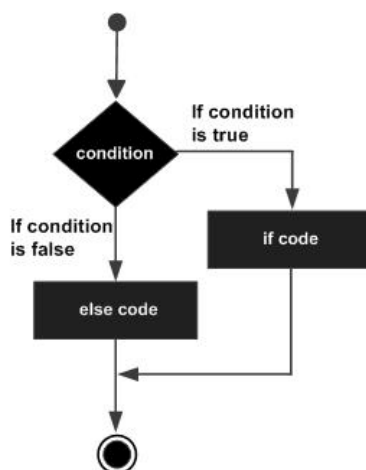


4.1. ESTRUTURAS CONDICIONAIS

As estruturas condicionais permitem executar diferentes blocos de código com base em condições específicas. Em Java, as principais estruturas condicionais são `if` e `switch`.

4.1.1. IF, ELSE IF, ELSE

A **estrutura if** avalia uma condição booleana e executa um bloco de código se a condição for verdadeira. As **cláusulas else if e else** permitem especificar condições alternativas ou um bloco padrão.



Sintaxe:

```

if (condicao) {
    // Código executado se a condição for verdadeira
} else if (outraCondicao) {
    // Código executado se a outra condição for verdadeira
} else {
    // Código executado se nenhuma condição anterior for verdadeira
}
  
```

EXEMPLO

```

public class CondicionalIf {
    public static void main(String[] args) {
        int idade = 20;
        if (idade >= 18) {
            System.out.println("Maior de idade");
        } else if (idade >= 13) {
            System.out.println("Adolescente");
        } else {
            System.out.println("Criança");
        }
    }
}
  
```

Saída: Maior de idade.

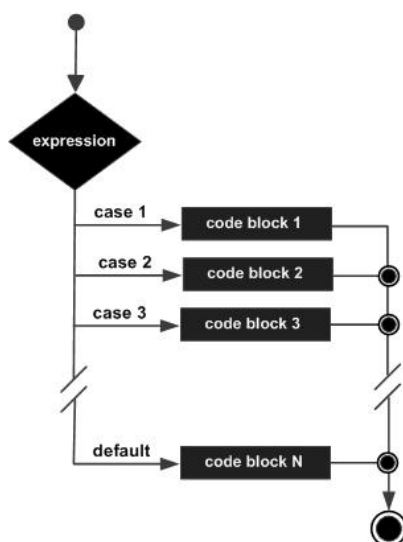
Obs.: A condição dentro do if deve resultar em um valor booleano (true ou false). Blocos com uma única instrução podem omitir as chaves {}.

Pergunta: Qual será a saída do código acima se idade=15?

Resposta: Adolescente.

4.1.2. SWITCH-CASE

A **instrução switch** pode ser usada quando várias instruções if-else são necessárias. Ele pode ter vários blocos de código junto com os valores de caso e executar um dos muitos blocos de código com base no valor do caso correspondente.



Sintaxe:

```

switch (expressao) {
    case valor1:
        // Código
        break;
    case valor2:
        // Código
        break;
    default:
        // Código padrão
}
  
```

EXEMPLO

```
public class CondicionalSwitch {
    public static void main(String[] args) {
        int dia = 3;
        switch (dia) {
            case 1:
                System.out.println("Domingo");
                break;
            case 2:
                System.out.println("Segunda-feira");
                break;
            case 3:
                System.out.println("Terça-feira");
                break;
            default:
                System.out.println("Dia inválido");
        }
    }
}
```

Saída: Terça-feira

- Obs.:**
- 1:** O break interrompe a execução do switch, evitando que os casos seguintes sejam executados.
 - 2:** O default é opcional e executado quando nenhum case corresponde.
 - 3:** A expressão do switch pode ser int, char, string ou enum.

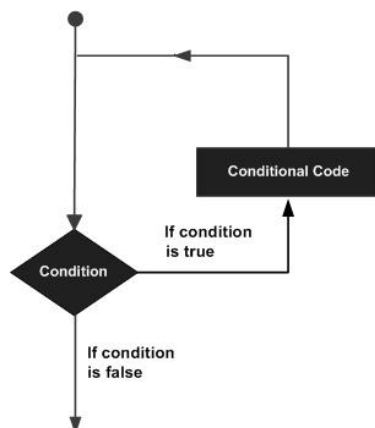
Pergunta: O que acontece se o break for omitido em um case?

Resposta: A execução continua nos casos seguintes até encontrar um break ou o fim do switch (fall-through).

4.2. ESTRUTURAS DE REPETIÇÃO

As estruturas de repetição ou loops permitem executar um bloco de códigos várias vezes, com base em uma condição ou contador. Java oferece três tipos principais: for, while e do-while.

Uma instrução **de loop** nos permite executar uma instrução ou grupo de instruções várias vezes e a seguir é a forma geral de uma instrução de loop na maioria das linguagens de programação.



4.2.1. FOR

O **laço for** é uma estrutura de controle de repetição usada para executar um bloco de código um **número específico de vezes**. É particularmente útil quando o número de iterações é conhecido de antemão, tornando-se uma maneira eficiente de automatizar tarefas repetitivas. Ele inclui inicialização, condição e incremento.

Sintaxe:

```
for (inicializacao; condicao; incremento) {
    // Código
}
```

EXEMPLO

```
public class LacoFor {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Número: " + i);
        }
    }
}
```

Saída:

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

4.2.2. WHILE

O **laço while** executa um bloco enquanto uma **condição for verdadeira**. É usado quando o **número de iterações não é conhecido previamente**.

Sintaxe:

```
while (condicao) {
    // Código
}
```

EXEMPLO

```
public class LacoWhile {
    public static void main(String[] args) {
        int contador = 1;
        while (contador <= 3) {
            System.out.println("Contador: " + contador);
            contador++;
        }
    }
}
```

Saída:

```
Contador: 1
Contador: 2
Contador: 3
```

Pergunta: Qual é a diferença entre for e while?

Resposta: O for é usado para iterações com contador definido, enquanto o while é usado para iterações baseadas em condições gerais.

4.2.3. DO-WHILE

O **laço do-while** executa o bloco **pelo menos uma vez antes** de verificar a condição.

Sintaxe:

```
do {
    // Código
} while (condicao);
```

EXEMPLO

```
public class LacoDoWhile {
    public static void main(String[] args) {
        int x = 0;
        do {
            System.out.println("Valor de x: " + x);
            x++;
        } while (x < 2);
    }
}
```

Saída:

Valor de x: 0

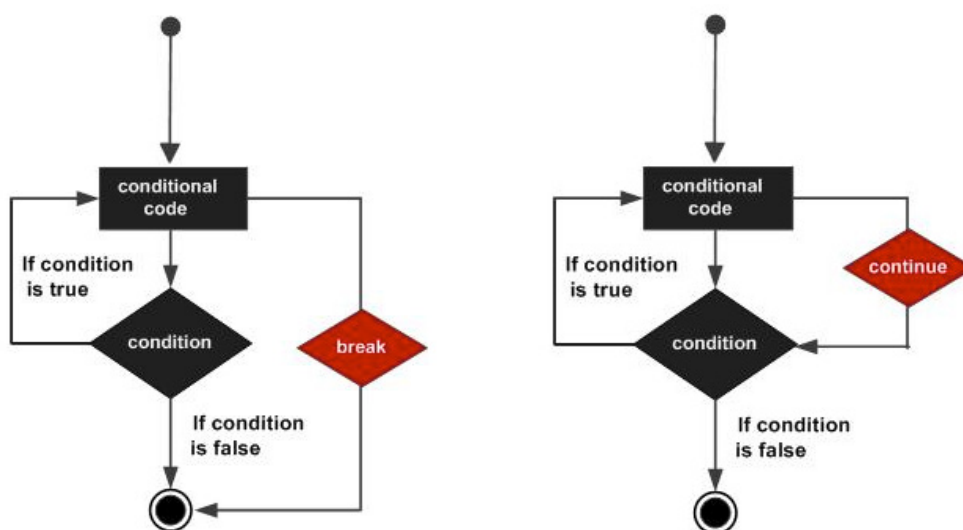
Valor de x: 1

Obs.: O do-while executa o bloco pelo menos uma vez, mesmo se a condição for falsa.

4.3. COMANDOS DE CONTROLE DE FLUXO

Os comandos break e continue alteram o fluxo de execução em laços e estruturas condicionais.

- **break:** interrompe a execução de um laço ou switch, saindo imediatamente;
- **continue:** pula para a próxima iteração de um laço, ignorando o restante do código no bloco atual.



EXEMPLO

```
public class ControleFluxo {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // Pula a iteração quando i == 3
            }
            if (i == 5) {
                break; // Sai do laço quando i == 5
            }
            System.out.println("i: " + i);
        }
    }
}
```

Saída:

```
i: 1
i: 2
i: 4
```

Obs.: Quando `i == 3`, o `continue` pula a impressão e quando `i == 5`, o `break` encerra o laço.

5. FUNÇÕES E MODULARIZAÇÃO

5.1. DEFINIÇÃO E CHAMADA DE MÉTODOS

Métodos Java são blocos de código que executam uma tarefa específica. Um método nos permite reutilizar o código, melhorando a eficiência e a organização. Todos os métodos em Java devem pertencer a uma **classe**. Os métodos são semelhantes às funções e expõem o comportamento dos objetos.

Sintaxe:

```
modificador tipoRetorno nomeDoMetodo(parametros) {
    // Código do método
}
```

EXEMPLO

```
public class Metodos {
    // Definição de um método
    public static void exibirSaudacao() {
        System.out.println("Olá, bem-vindo!");
    }
    public static void main(String[] args) {
        // Chamada do método
        exibirSaudacao();
    }
}
```

Saída: Olá, bem-vindo!

Obs.: **public static:** modificadores que definem acesso e contexto;
void: indica que o método não retorna valor;
exibirSaudacao(): nome do método, chamado diretamente no main.

5.1.1. AS VANTAGENS DOS MÉTODOS

- **Reusability:** os métodos nos permitem escrever código uma vez e usá-lo muitas vezes;

Obs.: Os métodos nos permitem abstrair a lógica complexa e fornecer uma interface simples para os outros usarem.

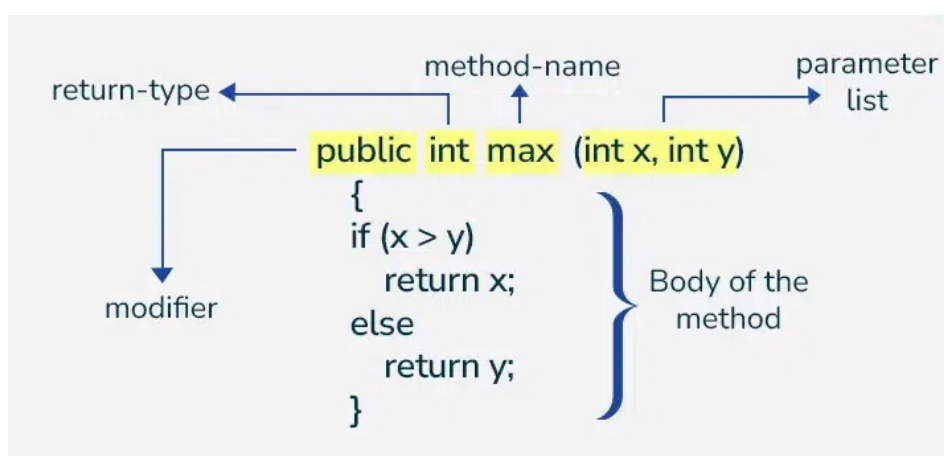
- **Encapsulamento:** permitir encapsular lógica e dados complexos;
- **Modularidade:** os métodos permitem-nos dividir o seu código em unidades menores e mais gerenciáveis, melhorando a modularidade do seu código;
- **Personalização:** fácil personalizar para tarefas específicas;
- **Desempenho melhorado:** ao organizar seu código em métodos bem estruturados, você pode melhorar o desempenho reduzindo a quantidade de código.

5.1.2. COMPONENTES-CHAVE DE UMA DECLARAÇÃO DO MÉTODO

- **Modificador:** especifica o nível de acesso do método (por exemplo, público, privado, protegido ou padrão);
- **Tipo de retorno:** o tipo de valor retornado ou anulado se nenhum valor for retornado;

- **Método nome:** segue as convenções de nomenclatura de Java; deve começar com um verbo minúsculo e usar o caso do camelo para várias palavras;
- **Parâmetros:** uma lista de valores de entrada (opcional). Parênteses vazios são usados se não forem necessários parâmetros;
- **Lista de exceções:** as exceções o método pode lançar (opcional);
- **Método corpo:** contém a lógica a ser executada (opcional no caso de métodos abstratos).

Componentes de um método



5.2. PARÂMETROS E RETORNO DE MÉTODOS

Métodos podem receber **parâmetros** (dados de entrada) e retornar valores usando a palavra-chave `return`.

- **Parâmetros:** variáveis declaradas na definição do método, que recebem valores quando o método é chamado;
- **Retorno:** um método pode retornar um valor de um tipo específico (ex.: `int`, `String`) ou nada (`void`).

EXEMPLO

```
public class Calculadora {
    // Método com parâmetros e retorno
    public static int somar(int a, int b) {
        return a + b;
    }
}
```

```
public static void main(String[] args) {
    int resultado = somar(5, 3); // Chamada com argumentos 5 e 3
    System.out.println("Soma: " + resultado);
}
}
```

Saída: Soma: 8

Obs.: 1:

- int a, int b: parâmetros do método somar;
- return a + b: retorna o resultado da soma;
- Na chamada somar (5, 3), os valores 5 e 3 são passados como argumentos.

2: O tipo de retorno deve corresponder ao valor retornado. Se for void, não usa return com valor.

5.3. SOBRECARGA DE MÉTODOS

Sobrecarga (**overloading**) permite definir múltiplos métodos com o mesmo nome, desde que tenham listas de parâmetros diferentes (em número ou tipo). Isso aumenta a flexibilidade do código.

Regras:

- O nome do método deve ser igual;
- Os parâmetros devem diferir em tipo, número ou ordem;
- O tipo de retorno não afeta a sobrecarga.

EXEMPLO

```
public class CalculadoraSobrecarga {
    // Método com dois inteiros
    public static int somar(int a, int b) {
        return a + b;
    }

    // Sobrecarga com três inteiros
    public static int somar(int a, int b, int c) {
        return a + b + c;
    }
}
```

```
// Sobrecarga com dois doubles
    public static double somar(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("Soma de inteiros: " + somar(2, 3));
        System.out.println("Soma de três inteiros: " + somar(2, 3, 4));
        System.out.println("Soma de doubles: " + somar(2.5, 3.7));
    }
}
```

Saída:

Soma de inteiros: 5

Soma de três inteiros: 9

Soma de doubles: 6.2

5.4. ESCOPO DE VARIÁVEIS

O escopo de uma variável determina onde ela pode ser acessada no programa. Em Java, o escopo é definido pelo bloco em que a variável é declarada.

Tipos de escopo:

- **Variáveis locais:** declaradas dentro de um método ou bloco, só podem ser acessadas nesse bloco;
- **Parâmetros:** escopo limitado ao método em que são definidos;
- **Variáveis de instância (globais):** declaradas na classe, fora de métodos, acessíveis por todos os métodos da classe.

EXEMPLO

```
public class Escopo {
    // Variável de instância (global)
    static int global = 100;

    public static void main(String[] args) {
        // Variável local
        int local = 10;
        System.out.println("Local: " + local);
        System.out.println("Global: " + global);
        testarEscopo();
    }
}
```

```
public static void testarEscopo() {
    // System.out.println(local); // Erro: local não está no escopo
    System.out.println("Global: " + global); // Acessível
}
}
```

Saída:

Local: 10

Global: 100

Global: 100

– local só existe dentro do método main;

– global é acessível em toda a classe porque é uma variável estática (usada aqui para simplificar; detalhes no Capítulo 6);

– Tentar acessar local em testarEscopo causa erro de compilação.

6. ESTRUTURAS DE DADOS BÁSICAS

Estruturas são usadas para armazenar e manipular conjuntos de dados. As estruturas de dados básicos em Java são **arrays**, **strings** e **arraylists**.

6.1. ARRAYS

Um array é uma estrutura de dados que armazena uma coleção de elementos do mesmo tipo em posições indexadas, começando pelo índice 0. Arrays têm **tamanho fixo, definido na criação**.

6.1.1. DECLARAÇÃO E INICIALIZAÇÃO

- **Declaração:** especifica o tipo de dados e o nome do array;
- **Inicialização:** define os elementos ou o tamanho do array.

Sintaxe:

```
tipo[] nomeDoArray; // Declaração
```

```
nomeDoArray = new tipo[tamanho]; // Inicialização
```

```
// Ou
```

```
tipo[] nomeDoArray = {valor1, valor2, ...}; // Declaração e inicialização
```

EXEMPLO

```
public class Arrays {
    public static void main(String[] args) {
        // Declaração e inicialização separadas
        int[] numeros = new int[3];
        numeros[0] = 10;
        numeros[1] = 20;
        numeros[2] = 30;

        // Declaração e inicialização direta
        String[] nomes = {"Ana", "Bruno", "Clara"};

        System.out.println("Primeiro número: " + numeros[0]);
        System.out.println("Segundo nome: " + nomes[1]);
    }
}
```

Saída:

Primeiro número: 10

Segundo nome: Bruno

- Obs.:** 1: O índice começa em 0 e vai até -1.
2: Acessar um índice inválido causa `ArrayIndexOutOfBoundsException`.

6.1.2. ACESSO E MANIPULAÇÃO DE ELEMENTOS

Os elementos de um array são acessados pelo índice e podem ser modificados diretamente. O atributo `length` retorna o tamanho do array.

EXEMPLO

```
public class ManipulacaoArray {
    public static void main(String[] args) {
        int[] valores = {1, 2, 3, 4, 5};
        System.out.println("Tamanho do array: " + valores.length);
        // Modificando um elemento
        valores[2] = 10;
    }
}
```

```
// Percorrendo o array
for (int i = 0; i < valores.length; i++) {
    System.out.println("Elemento " + i + ": " + valores[i]);
}
}
```

Saída:

Tamanho do array: 5

Elemento 0: 1

Elemento 1: 2

Elemento 2: 10

Elemento 3: 4

Elemento 4: 5

6.1.3. ARRAYS MULTIDIMENSIONAIS

Um **array multidimensional** é um array de arrays, comumente usado para representar matrizes (arrays bidimensionais, como tabelas com linhas e colunas) ou estruturas mais complexas (tridimensionais ou superiores). Em concursos, o foco é geralmente em matrizes bidimensionais.

Declaração e inicialização:

```
tipo[][] nomeDoArray; // Declaração de array bidimensional
nomeDoArray = new tipo[linhas][colunas]; // Inicialização
// Ou
tipo[][] nomeDoArray = {{valor1, valor2}, {valor3, valor4}}; // Declaração
e inicialização
```

EXEMPLO

```
public class ArrayMultidimensional {
    public static void main(String[] args) {
        // Declaração e inicialização de uma matriz 2x3
        int[][] matriz = new int[2][3];
        matriz[0][0] = 1;
        matriz[0][1] = 2;
        matriz[0][2] = 3;
        matriz[1][0] = 4;
        matriz[1][1] = 5;
        matriz[1][2] = 6;
    }
}
```

```
// Inicialização direta
int[][] outraMatriz = {{10, 20, 30}, {40, 50, 60}};

// Acessando elementos
System.out.println("Elemento [0][1]: " + matriz[0][1]);
System.out.println("Tamanho (linhas): " + matriz.length);
System.out.println("Tamanho (colunas da linha 0): " + matriz[0].length);

// Percorrendo a matriz
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

Saída:

Elemento [0][1]: 2

Tamanho (linhas): 2

Tamanho (colunas da linha 0): 3

1 2 3

4 5 6

- Obs.:**
- 1: `matriz.length` retorna o número de linhas.
 - 2: `matriz[i].length` retorna o número de colunas da linha `i`.
 - 3: Arrays multidimensionais podem ser "irregulares" (linhas com tamanhos diferentes), mas concursos geralmente usam matrizes regulares.
 - 4: Acessar índices inválidos causa `ArrayIndexOutOfBoundsException`.

6.2. STRINGS

Uma **String** em Java é uma classe que representa sequências de caracteres.

6.2.1. MÉTODOS COMUNS

A classe String possui métodos úteis para manipulação de texto. Alguns dos mais comuns são:

- **length():** retorna o número de caracteres;
- **substring(int inicio, int fim):** extrai uma parte da string (do índice inicio até fim-1);
- **toUpperCase():** converte para maiúsculas;
- **toLowerCase():** converte para minúsculas;
- **charAt(int indice):** retorna o caractere no índice especificado;
- **equals(String outra):** compara duas strings quanto ao conteúdo.

EXEMPLO

```
public class Strings {
    public static void main(String[] args) {
        String texto = "Java para Concursos";
        System.out.println("Tamanho: " + texto.length());
        System.out.println("Subtring: " + texto.substring(0, 4));
        System.out.println("Maiúsculas: " + texto.toUpperCase());
        System.out.println("Caractere na posição 2: " + texto.charAt(2));
        System.out.println("Igual a 'Java': " + texto.equals("Java"));
    }
}
```

Saída:

Tamanho: 19

Subtring: Java

Maiúsculas: JAVA PARA CONCURSOS

Caractere na posição 2: v

Igual a 'Java': false

Obs.: **Strings em Java são imutáveis**, ou seja, métodos como toUpperCase criam uma nova string, sem alterar a original.

6.2.2. MANIPULAÇÃO DE STRINGS

Strings podem ser concatenadas com o operador + ou manipuladas com métodos adicionais, como replace e trim.

EXEMPLO

```
public class ManipulacaoString {
    public static void main(String[] args) {
        String frase = " Aprenda Java! ";
        System.out.println("Concatenação: " + frase + " Hoje!");
        System.out.println("Sem espaços: " + frase.trim());
        System.out.println("Substituir 'Java': " + frase.replace("Java",
"Programação"));
    }
}
```

Saída:

Concatenação: Aprenda Java! Hoje!

Sem espaços: Aprenda Java!

Substituir 'Java': Aprenda Programação!

6.3. COLEÇÕES (ARRAYLIST)

O **ArrayList** é uma classe da biblioteca Java (java.util) que implementa uma **lista dinâmica**, permitindo adicionar ou remover elementos sem tamanho fixo, ao contrário dos arrays.

O ArrayList é parte do **Java Collections Framework**, que também inclui outras estruturas, como List (listas ordenadas), Set (elementos únicos) e Map (mapeamentos chave-valor).

6.3.1. CRIAÇÃO E MANIPULAÇÃO BÁSICA

Principais métodos:

- **add(elemento)**: adiciona um elemento à lista;
- **get(indice)**: retorna o elemento no índice especificado;
- **size()**: retorna o número de elementos;
- **remove(indice)**: remove o elemento no índice.

EXEMPLO

```
import java.util.ArrayList;

public class ArrayListExemplo {
    public static void main(String[] args) {
        ArrayList<String> nomes = new ArrayList<>();
        nomes.add("Ana");
        nomes.add("Bruno");
        nomes.add("Clara");
    }
}
```

```
System.out.println("Tamanho: " + nomes.size());
System.out.println("Primeiro nome: " + nomes.get(0));

nomes.remove(1); // Remove "Bruno"
System.out.println("Após remoção: " + nomes);
    }
}
```

Saída:

Tamanho: 3

Primeiro nome: Ana

Após remoção: [Ana, Clara]

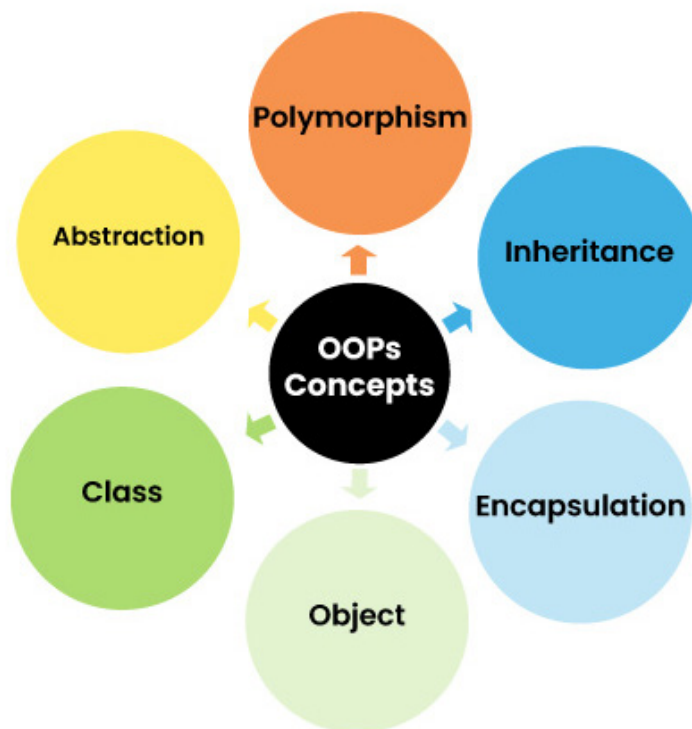
6.3.2. DIFERENÇAS ENTRE ARRAYS E ARRAYLIST

Característica	Array	ArrayList
Tamanho	Fixo	Dinâmico (pode crescer)
Tipo de dados	Primitivos ou objetos	Apenas objetos
Métodos disponíveis	Poucos (ex.: length)	Muitos (ex.: add, remove)
Declaração	int[] arr = new int[5];	ArrayList<Integer> list = new ArrayList<>();

Resumindo, **arrays têm tamanho fixo, enquanto arraylists podem crescer dinamicamente.**

7. PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)

Programação Orientada a Objetos (POO) é um paradigma de programação que organiza o código em torno de **objetos**, que são estruturas que combinam dados (**atributos**) e comportamentos (**métodos**). A ideia central é modelar elementos do mundo real no software, facilitando o desenvolvimento, a manutenção e a reutilização do código. Seus quatro pilares principais são: **Abstração, Encapsulamento, Herança e Polimorfismo.**



7.1. CONCEITOS FUNDAMENTAIS

7.1.1. CLASSES E OBJETOS

Na programação orientada a objetos, uma **classe** é um modelo a partir do qual objetos individuais são criados (ou, podemos dizer que uma classe é um tipo de dados de um tipo de objeto). Em Java, tudo está relacionado a classes e objetos. Cada classe tem seus métodos e atributos que podem ser acessados e manipulados através dos objetos.

Já um **objeto** é uma entidade que tem duas características (estados e comportamento). Alguns dos objetos do mundo real são livro, celular, mesa, computador etc. Um objeto é uma **variável da classe tipo**, é um componente básico de um sistema de programação orientado a objetos. Uma classe tem os métodos e os membros dos dados (atributos), esses métodos e membros de dados são acessados através de um objeto. Assim, **um objeto é uma instância de uma classe**.

EXEMPLO

```

public class Pessoa {
    // Atributos
    String nome;
    int idade;
}
  
```

```
// Método
void exibirInformacoes() {
    System.out.println("Nome: " + nome + ", Idade: " + idade);
}
}

class TestePessoa {
    public static void main(String[] args) {
        // Criando um objeto
        Pessoa pessoa = new Pessoa();
        pessoa.nome = "Ana";
        pessoa.idade = 25;
        pessoa.exibirInformacoes();
    }
}
```

Saída: Nome: Ana, Idade: 25

- Obs.:**
- 1:** Pessoa é a classe com atributos nome e idade, e método exibirinformacoes.
 - 2:** New Pessoa () cria um objeto da classe Pessoa.
 - 3:** Os atributos do objeto são acessados com o operador..

7.1.2. ATRIBUTOS E MÉTODOS

- **Atributos:** variáveis definidas na classe, representando as propriedades de um objeto;
- **Métodos:** funções que definem o comportamento do objeto, manipulando seus atributos ou executando ações.

EXEMPLO

```
public class Carro {
    String modelo;
    int velocidade;

    void acelerar(int incremento) {
        velocidade += incremento;
    }
}
```

```
class TesteCarro {
    public static void main(String[] args) {
        Carro carro = new Carro();
        carro.modelo = "Fusca";
        carro.velocidade = 50;
        carro.acelerar(20);
        System.out.println("Modelo: " + carro.modelo + ", Velocidade: " +
carro.velocidade);
    }
}
```

Saída: Modelo: Fusca, Velocidade: 70

Pergunta: O que define o comportamento de um objeto em Java?

Resposta: Métodos

7.1.3. CONSTRUTORES

Um **construtor** é um método especial usado para inicializar objetos. Ele tem o mesmo nome da sua classe e é sintaticamente semelhante a um método. Não retorna valor (nem void).

Normalmente, você usará um construtor para dar valores iniciais às variáveis de instância definidas pela classe ou para executar quaisquer outros procedimentos de inicialização necessários para criar um objeto totalmente formado.

Todas as classes têm construtores, quer você defina uma ou não porque o Java fornece automaticamente um construtor padrão que inicializa todas as variáveis de membro para zero. No entanto, uma vez que você define seu construtor, o construtor padrão não é mais usado.

Regras para a criação de construtores Java:

- O nome dos construtores deve ser o mesmo que o nome da classe;
- Os construtores Java não têm um tipo de retorno. Nem mesmo use o vazio como um tipo de retorno;
- Pode haver vários construtores na mesma classe, este conceito é conhecido como superação de construtores;
- Os modificadores de acesso podem ser usados com os construtores, use se você quiser alterar a visibilidade/acessibilidade dos construtores;
- Java fornece um construtor padrão que é invocado durante o tempo de criação de objeto. Se você criar qualquer tipo de construtor, o construtor padrão (fornecido por Java) não é invocado.

EXEMPLO

```
public class Estudante {
    String nome;
    int matricula;

    // Construtor
    public Estudante(String nome, int matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }

    void exibirDados() {
        System.out.println("Nome: " + nome + ", Matrícula: " + matricula);
    }
}

class TesteEstudante {
    public static void main(String[] args) {
        Estudante estudante = new Estudante("Bruno", 12345);
        estudante.exibirDados();
    }
}
```

Saída: Nome: Bruno, Matrícula: 12345

Notas:

- this diferencia atributos da classe de parâmetros com mesmo nome;
- Se nenhum construtor for definido, Java fornece um construtor padrão sem parâmetros.

Pergunta: Qual é a função de um construtor em Java?

Resposta: Inicializar objetos de uma classe.

7.2. ENCAPSULAMENTO

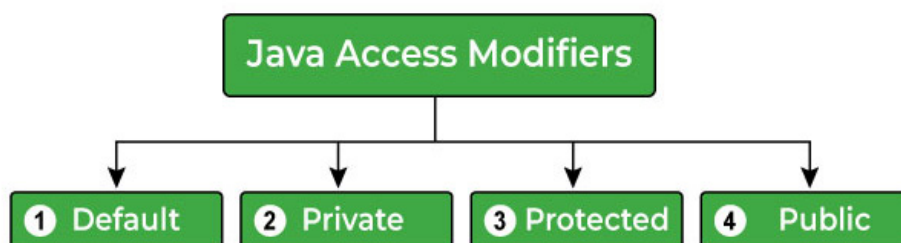
Encapsulamento protege os dados de uma classe, restringindo o acesso direto aos atributos e fornecendo métodos para manipulá-los (getters e setters). É um mecanismo de envolver os dados (variáveis) e código agindo sobre os dados (métodos) juntos como uma única unidade. No encapsulamento, as variáveis de uma classe serão ocultadas de outras classes e podem ser acessadas apenas através dos métodos de sua classe atual. Por isso, também é conhecido como **esconder os dados**.

7.2.1. MODIFICADORES DE ACESSO (PUBLIC, PRIVATE, PROTECTED)

Modificadores de acesso Java são usados para especificar o escopo das variáveis, membros de dados, métodos, classes ou construtores. Estes ajudam a restringir e proteger o acesso (ou nível de acesso) dos dados.

Existem quatro tipos diferentes de modificadores de acesso em Java, nós os listamos da seguinte forma:

- Padrão (sem palavra-chave necessária);
- Privada;
- Protegido;
- O público.



EXEMPLO

```

public class ContaBancaria {
    private double saldo;

    // Getter
    public double getSaldo() {
        return saldo;
    }

    // Setter
    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }
}
  
```

7.2.2. GETTERS E SETTERS

Métodos get e set controlam o acesso aos atributos privados.

EXEMPLO

```
public class ContaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }

    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
        }
    }
}

class TesteConta {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria();
        conta.setSaldo(1000);
        conta.depositar(500);
        System.out.println("Saldo: " + conta.getSaldo());
    }
}
```

Saída: Saldo: 1500.0

Pergunta: Qual modificador de acesso restringe um atributo à própria classe?

Resposta: private

7.3. HERANÇA

A programação orientada a objetos, **herança** é um mecanismo que permite a **reutilização de código** por meio do relacionamento entre classes. Com ela, uma **classe derivada** (ou **subclasse**) pode herdar os **atributos e métodos** de uma **classe base** (ou **superclasse**), aproveitando suas funcionalidades já implementadas.

Esse conceito facilita a manutenção e a expansão do código, pois permite **adicionar novos comportamentos** à subclasse sem alterar a classe original. Além disso, promove a **especialização**, já que a subclasse pode estender ou sobrescrever os comportamentos herdados. Em Java, a herança é implementada com a palavra-chave **extends**.

7.3.1. USO DE EXTENDS

EXEMPLO

```
public class Animal {
    String nome;

    public void comer() {
        System.out.println(nome + " está comendo.");
    }
}

public class Cachorro extends Animal {
    public void latir() {
        System.out.println(nome + " está latindo.");
    }
}

class TesteHeranca {
    public static void main(String[] args) {
        Cachorro cachorro = new Cachorro();
        cachorro.nome = "Rex";
        cachorro.comer();
        cachorro.latir();
    }
}
```

Saída:

Rex está comendo.

Rex está latindo.

7.3.2. SOBRESCRITA DE MÉTODOS

Uma subclasse pode redefinir um método da superclasse usando a anotação `@Override`.

EXEMPLO

```
public class Animal {
    public void fazerSom() {
        System.out.println("Som genérico");
    }
}

public class Gato extends Animal {
    @Override
    public void fazerSom() {
        System.out.println("Miau");
    }
}

class TesteSobrescrita {
    public static void main(String[] args) {
        Gato gato = new Gato();
        gato.fazerSom();
    }
}
```

Saída: Miau

Pergunta: Qual anotação é usada para indicar sobrescrita de métodos?

Resposta: `@Override`

7.4. POLIMORFISMO

O polimorfismo é comumente implementado em Java usando métodos de sobreposição (métodos com a mesma assinatura, mas comportamentos diferentes nas classes superclasse e subclasse) e interfaces.

Polimorfismo permite que objetos de diferentes classes sejam tratados como objetos de uma classe comum, geralmente via herança ou interfaces.

7.4.1. POLIMORFISMO DE SOBRECARGA E SOBRESCRITA

- **Sobrecarga (Tempo de compilação):** métodos com mesmo nome, mas parâmetros diferentes;
- **Sobrescrita (Tempo de execução):** redefinição de métodos da superclasse.

EXEMPLO

```
public class Animal {
    public void fazerSom() {
        System.out.println("Som genérico");
    }
}

public class Cachorro extends Animal {
    @Override
    public void fazerSom() {
        System.out.println("Au au");
    }
}

class TestePolimorfismo {
    public static void main(String[] args) {
        Animal animal = new Cachorro(); // Polimorfismo
        animal.fazerSom();
    }
}
```

Saída: Au au

Nota: O método chamado depende do tipo do objeto, não da referência.

Pergunta: Qual é o resultado de chamar fazerSom() em uma referência Animal apontando para um objeto Cachorro?

Resposta: Au au

7.5. CLASSES E MÉTODOS ABSTRATOS

Uma **classe abstrata** não pode ser instanciada e é usada como base para outras classes. Um **método abstrato** não tem implementação e deve ser sobrescrito pelas subclasses.

EXEMPLO

```
public abstract class Forma {
    public abstract double calcularArea();
}

public class Circulo extends Forma {
    private double raio;

    public Circulo(double raio) {
        this.raio = raio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }
}

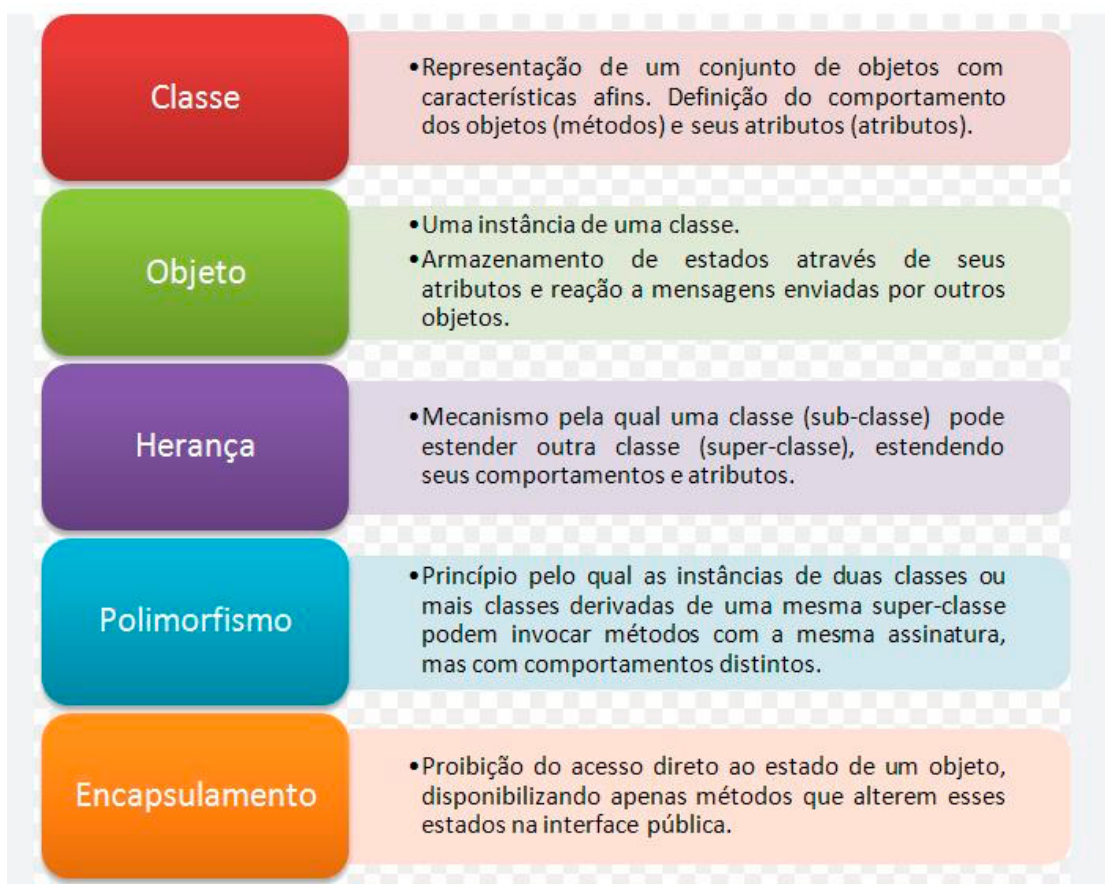
class TesteAbstrato {
    public static void main(String[] args) {
        Forma circulo = new Circulo(5);
        System.out.println("Área: " + circulo.calcularArea());
    }
}
```

Saída: Área: 78.53981633974483

Pergunta: Uma classe abstrata pode ser instanciada?

Resposta: Não

Resumindo:



DIRETO DO CONCURSO

001. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Na linguagem de programação *Java*, quando uma classe é abstrata:

- a) não se pode realizar a sobrecarga de construtores.
- b) o modificador de acesso deve ser o *protected*.
- c) não pode ser instanciada diretamente.
- d) ela não pode ser herdada.
- e) todos os métodos devem ser abstratos.



a) Errada. Classes abstratas podem ter construtores e podem ter sobrecarga de construtores normalmente. A única limitação é que não podem ser instanciadas diretamente, mas seus construtores são chamados pelas subclasses.

- b) Errada. Não há exigência de que o modificador de uma classe abstrata seja *protected*. Ela pode ser *public*, *protected*, ou sem modificador (package-private), conforme necessário. *Protected* é restrito a membros (atributos e métodos), não a classes top-level.
- c) Certa. Esta é a característica principal de uma classe abstrata; não pode ser instanciada diretamente. Ela serve como base para outras classes que a estendem.
- d) Errada. Classes abstratas são feitas para serem herdadas. Esse é um dos principais usos da abstração: criar uma estrutura comum que será estendida por subclasses concretas.
- e) Errada. Uma classe abstrata pode conter métodos abstratos ou não. Ela pode ter métodos com implementação completa. Apenas classes que são 100% abstratas (interfaces até o Java 7) tinham essa restrição.

Letra c.

7.6. INTERFACES

A interface Java é uma coleção de métodos abstratos. A interface é usada para obter **abstração** na qual você pode definir métodos sem suas implementações (sem ter o corpo dos métodos). Uma interface é um tipo de referência e é **semelhante à classe**.

Juntamente com métodos abstratos, uma interface também pode conter constantes, métodos padrão, métodos estáticos e tipos aninhados. Os corpos dos métodos existem apenas para métodos padrão e métodos estáticos.

Escrever uma interface é semelhante a escrever uma classe. No entanto, uma classe descreve os atributos e comportamentos de um objeto. Uma interface contém comportamentos que uma classe implementa. A menos que a classe que implementa a interface seja abstrata, todos os métodos da interface precisam ser definidos na classe.

As interfaces têm as seguintes propriedades:

- Uma interface é implicitamente abstrata. Você não precisa usar a palavra-chave **abstrata** ao declarar uma interface;
- Cada método em uma interface também é implicitamente abstrato, portanto, a palavra-chave abstrata não é necessária;
- Os métodos em uma interface são implicitamente públicos.

7.6.1. INTERFACES E CLASSES JAVA: SEMELHANÇAS E DIFERENÇAS

Uma **interface** é **semelhante** a uma **classe** das seguintes maneiras:

- Uma interface pode conter qualquer número de métodos;
- Uma interface é escrita em um arquivo com uma extensão.java, com o nome da interface correspondente ao nome do arquivo;

- O código de byte de uma interface aparece em um arquivo.class;
- As interfaces aparecem em pacotes e seu arquivo bytecode correspondente deve estar em uma estrutura de diretório que corresponda ao nome do pacote.

No entanto, uma **interface** é **diferente** de uma **classe** de várias maneiras, incluindo

- Você não pode instanciar uma interface;
- Uma interface não contém construtores;
- Todos os métodos em uma interface são abstratos;
- Uma interface não pode conter campos de instância. Os únicos campos que podem aparecer em uma interface devem ser declarados estáticos e finais;
- Uma interface não é estendida por uma classe; ela é implementada por uma classe;
- Uma interface pode estender várias interfaces.

7.6.2. IMPLEMENTAÇÃO DE INTERFACE

Uma classe implementa uma interface em Java utilizando a palavra-chave **implements**. Ao fazer isso, ela se compromete a fornecer implementações para todos os métodos declarados na interface. Caso não implemente todos esses métodos, a classe deve ser declarada como abstrata.

Interfaces definem contratos que promovem **flexibilidade** e **polimorfismo**, permitindo que diferentes classes compartilhem comportamentos comuns sem estarem diretamente relacionadas por herança.

Regras para definir métodos imperiosos em interfaces Java:

- As exceções verificadas não devem ser declaradas em métodos de execução diferentes dos declarados pelo método de interface ou subclasses dos declarados pelo método de interface;
- A assinatura do método da interface e o mesmo tipo ou subtipo de retorno devem ser mantidos ao substituir os métodos;
- Uma classe de implementação em si pode ser abstrata e, em caso afirmativo, os métodos de interface não precisam ser implementados.

Regras para a implementação de interfaces Java:

- Uma classe pode implementar mais de uma interface de cada vez;
- Uma classe pode estender apenas uma classe, mas implementar muitas interfaces;
- Uma interface pode estender outra interface, de forma semelhante, uma vez que uma classe pode estender outra classe.

EXEMPLO

```
public interface Veiculo {
    void mover();
}

public class Carro implements Veiculo {
    @Override
    public void mover() {
        System.out.println("Carro está se movendo.");
    }
}

class TesteInterface {
    public static void main(String[] args) {
        Veiculo veiculo = new Carro();
        veiculo.mover();
    }
}
```

Saída: Carro está se movendo.

Obs.: 1: Usa-se implements para implementar uma interface.

2: Uma classe pode implementar múltiplas interfaces.

3: Métodos de uma interface são implicitamente public e abstract (antes do Java 8).

Pergunta: Qual palavra-chave é usada para implementar uma interface?

Resposta: implements

8. TRATAMENTO DE EXCEÇÕES

O tratamento de exceções em Java é um mecanismo essencial para lidar com erros durante a execução de um programa. Exceções permitem que o programa gerencie situações inesperadas, como divisões por zero ou acesso a índices inválidos.

Uma exceção pode ocorrer por vários motivos. A seguir estão alguns cenários em que ocorre uma exceção.

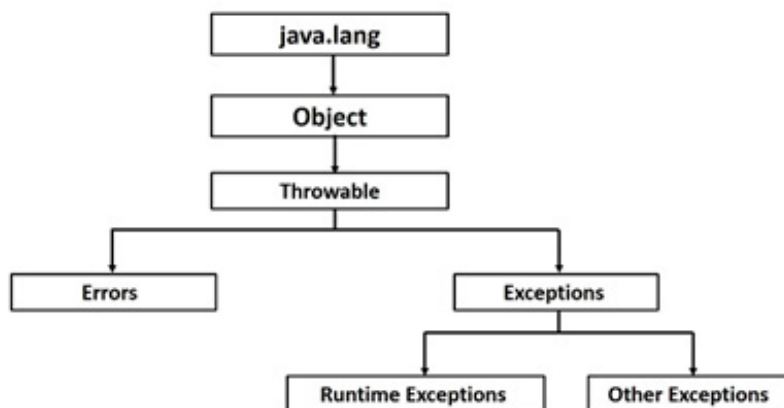
- Um usuário inseriu dados inválidos;
- Um arquivo que precisa ser aberto não pode ser encontrado;
- Uma conexão de rede foi perdida no meio das comunicações ou a JVM ficou sem memória.

Algumas dessas exceções são causadas por erro do usuário, outras por erro do programador e outras por recursos físicos que falharam de alguma forma.

8.1. CONCEITO

Uma **exceção** é um evento anormal que ocorre durante a execução de um programa, interrompendo o fluxo normal. Em Java, exceções são objetos da classe Throwable, divididos em duas categorias:

- **Error**: problemas graves, geralmente fora do controle do programador (ex.: OutOfMemoryError). Não é comum tratá-los;
- **Exception**: problemas que podem ser tratados pelo programador, divididos em:
 - **Checked Exceptions** (ex.: IOException): é uma exceção que é verificada (notificada) pelo compilador em tempo de compilação, estes também são chamados de **exceções de tempo de compilação**. Essas exceções não podem ser simplesmente ignoradas, o programador deve cuidar dessas exceções. Devem ser tratadas ou declaradas no método com throws;
 - **Unchecked Exceptions** (ex.: NullPointerException): não exigem tratamento explícito. Uma exceção não verificada é uma exceção que ocorre no momento da execução. Estes também são chamados de **Exceções de Tempo de Execução**. Estes incluem erros de programação, como erros de lógica ou uso indevido de uma API. As exceções de tempo de execução são ignoradas no momento da compilação.



EXEMPLO

```

public class ExemploExcecao {
    public static void main(String[] args) {
        int[] array = {1, 2, 3};
        System.out.println(array[5]); // Causa ArrayIndexOutOfBoundsException
    }
}
    
```

Saída: Gera uma exceção, pois o índice 5 está fora dos limites do array.

8.2. BLOCOS TRY-CATCH

O bloco **try-catch** é usado para capturar e tratar exceções, evitando que o programa termine abruptamente.

Um método captura uma exceção usando uma combinação da **tentativa** e **captura** de palavras-chave. Um bloco de tentativa e captura é colocado em torno do código que pode gerar uma exceção. O código dentro de um bloco de tentativa e captura é referido como código protegido.

Sintaxe:

```
try {
    // Código que pode gerar exceção
} catch (TipoDeExcecao variavel) {
    // Tratamento da exceção
} finally {
    // Código executado sempre (opcional)
}
```

EXEMPLO

```
public class TryCatch {
    public static void main(String[] args) {
        try {
            int divisao = 10 / 0; // Causa ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Erro: Divisão por zero!");
        } finally {
            System.out.println("Bloco finally executado.");
        }
    }
}
```

Saída:

Erro: Divisão por zero!

Bloco finally executado.

Obs.: 1: O bloco try contém o código que pode gerar uma exceção.

2: O bloco catch especifica o tipo de exceção a ser tratada. Pode haver múltiplos catch para diferentes tipos.

3: O bloco finally (opcional) executa sempre, independentemente de uma exceção ocorrer ou não.

4: Use a classe base Exception para capturar qualquer exceção, mas isso é menos específico.

Pergunta: Qual bloco é executado mesmo se nenhuma exceção ocorrer?

Resposta: finally

8.3. LANÇAMENTO DE EXCEÇÕES

A palavra-chave **throw** é usada para lançar uma exceção explicitamente, enquanto **throws** declara que um método pode lançar exceções específicas.

EXEMPLO

throw

```
public class LancarExcecao {
    public static void verificarIdade(int idade) {
        if (idade < 18) {
            throw new IllegalArgumentException("Menor de idade!");
        }
        System.out.println("Acesso permitido.");
    }

    public static void main(String[] args) {
        try {
            verificarIdade(16);
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Saída: Erro: Menor de idade!

throws:

```
public class DeclararExcecao {
    public static void metodoComExcecao() throws Exception {
        throw new Exception("Exceção lançada!");
    }
}
```

```

public static void main(String[] args) {
    try {
        metodoComExcecao();
    } catch (Exception e) {
        System.out.println("Erro: " + e.getMessage());
    }
}
}

```

Saída: Erro: Exceção lançada!

Obs.: 1: throw cria e lança a exceção.

2: throws é usado na assinatura do método para indicar possíveis exceções checked.

8.4. EXCEÇÕES COMUNS

Algumas exceções são frequentemente cobradas em concursos devido ao seu uso comum:

- **NullPointerException:** ocorre ao tentar acessar um objeto ou método de uma referência nula;
- **ArrayIndexOutOfBoundsException:** ocorre ao acessar um índice inválido em um array;
- **ArithmeticException:** ocorre em operações matemáticas inválidas, como divisão por zero;
- **IllegalArgumentException:** lançada quando um argumento inválido é passado a um método;
- **ClassCastException:** ocorre ao tentar fazer um cast inválido de tipos.

EXEMPLO

```

public class ExcecoesComuns {
    public static void main(String[] args) {
        try {
            String texto = null;
            System.out.println(texto.length()); // NullPointerException
            int[] array = {1, 2};
            System.out.println(array[5]); // ArrayIndexOutOfBoundsException
            int divisao = 10 / 0; // ArithmeticException
        } catch (NullPointerException e) {
            System.out.println("Erro: Referência nula!");
        }
    }
}

```

```
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Erro: Índice inválido!");  
} catch (ArithmeticException e) {  
    System.out.println("Erro: Divisão por zero!");  
}  
}  
}
```

Saída: Erro: Referência nula!

Obs.: A ordem dos blocos catch é importante: exceções mais específicas devem vir antes das mais genéricas (ex.: NullPointerException antes de Exception).

Pergunta: Qual exceção é lançada ao tentar acessar o método de um objeto nulo?

Resposta: NullPointerException

RESUMO

INTRODUÇÃO AO JAVA

- Portável, robusto e seguro;
- Slogan: *Write Once, Run Anywhere (WORA)*;
- Sintaxe baseada em C/C++;
- Compilado para bytecode e executado pela JVM.

FUNDAMENTOS TÉCNICOS

- **JVM**: executa bytecode, otimiza com JIT e gerencia memória com o Garbage Collector;
- **Bytecode**: intermediário entre código Java e máquina real;
- **Garbage Collector**: libera automaticamente objetos não utilizados.

AMBIENTE DE DESENVOLVIMENTO

- **JDK**: inclui compilador, JVM, bibliotecas;
- IDEs populares: IntelliJ, Eclipse, NetBeans, VS Code.

SINTAXE E ELEMENTOS BÁSICOS

- **Classe**: estrutura principal;
- **Método main**: ponto de entrada;
- **Tipos de dados**:
 - Primitivos: int, float, boolean, char etc.;
 - Referência: String, arrays, objetos, interfaces;
- **Operadores**: aritméticos, relacionais, lógicos, atribuição, incremento.

ENTRADA E SAÍDA

- System.out.print, println, printf;
- Scanner: entrada de dados pelo usuário (nextInt(), nextLine() etc.).

ESTRUTURAS DE CONTROLE

- **Condicionais:** if, else if, else, switch;
- **Repetição:** for, while, do-while;
- **Controle de fluxo:** break, continue.

MÉTODOS (FUNÇÕES)

- **Declaração:** public static tipo nome(parâmetros);
- **Retorno:** com return;
- **Parâmetros:** entrada de dados;
- **Sobrecarga:** métodos com mesmo nome, mas assinaturas diferentes;
- **Escopo:** local, parâmetro, instância.

ESTRUTURAS DE DADOS

- **Arrays:**
 - Unidimensionais e multidimensionais;
 - Acesso via índice;
- **String:**
 - Imutáveis;
 - Métodos: length(), charAt(), substring(), toUpperCase() etc.;
- **ArrayList:** lista dinâmica (add(), remove(), get(), size()).

PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)

Conceitos principais:

- **Classe e objeto;**
- **Atributos** (estado) e **métodos** (comportamento);
- **Construtores:** inicializam objetos;
- **Encapsulamento:**
 - Modificadores: private, public, protected;
 - Getters e setters;
- **Herança:** extends permite reutilização;
- **Polimorfismo:**
 - Sobrecarga (em tempo de compilação);
 - Sobrescrita (em tempo de execução, com @Override);
- **Classes e métodos abstratos:** base para subclasses;

- **Interfaces:**
 - Definem contratos;
 - Implementadas com implements.

TRATAMENTO DE EXCEÇÕES

- **Tipos:**
 - Checked (por exemplo: IOException);
 - Unchecked (por exemplo: NullPointerException);
- **Blocos:**
 - try – código que pode lançar erro;
 - catch – tratamento;
 - finally – executado sempre;
- **Palavras-chave:** throw, throws.

Palavras-chave do Java mais comumente cobradas em concursos públicos.

Palavra-Chave	Descrição	Exemplo
abstract	Define uma classe ou método abstrato, que não pode ser instanciado diretamente ou deve ser implementado por subclasses.	java abstract class Forma { abstract void desenhar(); }
boolean	Tipo de dado primitivo que armazena valores true ou false.	java boolean ativo = true;
break	Interrompe a execução de um laço ou switch.	java for (int i = 0; i < 5; i++) { if (i == 3) break; System.out.println(i); }
byte	Tipo de dado primitivo para inteiros de 8 bits (-128 a 127).	java byte pequeno = 100;
catch	Define um bloco para capturar e tratar exceções.	java try { int x = 1 / 0; } catch (ArithmeticException e) { System.out.println("Erro!"); }
char	Tipo de dado primitivo para um caractere Unicode de 16 bits.	java char letra = 'A';
class	Define uma classe, que é o modelo para criar objetos.	java public class Pessoa { String nome; }
continue	Pula para a próxima iteração de um laço.	java for (int i = 0; i < 5; i++) { if (i == 2) continue; System.out.println(i); }
do	Inicia um laço do-while, que executa o bloco pelo menos uma vez.	java int i = 0; do { System.out.println(i); i++; } while (i < 3);

Palavra-Chave	Descrição	Exemplo
double	Tipo de dado primitivo para números decimais de 64 bits.	java double preco = 99.99;
else	Define o bloco alternativo em uma estrutura condicional if.	java if (x > 0) { System.out.println("Positivo"); } else { System.out.println("Negativo ou zero"); }
extends	Indica que uma classe herda de outra classe.	java class Estudante extends Pessoa { int matricula; }
float	Tipo de dado primitivo para números decimais de 32 bits.	java float altura = 1.75f;
for	Define um laço de repetição com inicialização, condição e incremento.	java for (int i = 0; i < 5; i++) { System.out.println(i); }
if	Define uma estrutura condicional para executar um bloco se a condição for verdadeira.	java if (idade >= 18) { System.out.println("Maior de idade"); }
implements	Indica que uma classe implementa uma interface.	java interface Animal { void fazerSom(); } class Cachorro implements Animal { public void fazerSom() { System.out.println("Au au"); } }
import	Importa classes ou pacotes para uso no programa.	java import java.util.Scanner;
int	Tipo de dado primitivo para números inteiros de 32 bits.	java int idade = 25;
long	Tipo de dado primitivo para inteiros de 64 bits.	java long distancia = 1000000L;
new	Cria um novo objeto ou aloca memória para arrays.	java String texto = new String("Olá");
private	Modificador de acesso que restringe o acesso a membros de uma classe.	java class Pessoa { private String nome; }
protected	Modificador de acesso que permite acesso dentro do mesmo pacote ou em subclasses.	java class Pessoa { protected String endereco; }
public	Modificador de acesso que permite acesso irrestrito a membros ou classes.	java public class Programa { public static void main(String[] args) {} }
return	Retorna um valor de um método ou encerra sua execução.	java int somar(int a, int b) { return a + b; }
short	Tipo de dado primitivo para inteiros de 16 bits.	java short contador = 1000;

Palavra-Chave	Descrição	Exemplo
static	Indica que um membro pertence à classe, não a instâncias.	java class Calculadora { static int somar(int a, int b) { return a + b; }
switch	Inicia uma estrutura de seleção múltipla.	java switch (dia) { case 1: System.out.println("Domingo"); break; default: System.out.println("Outro dia"); }
throw	Lança uma exceção explicitamente.	java if (idade < 0) { throw new IllegalArgumentException("Idade inválida"); }
throws	Declara que um método pode lançar exceções.	java void verificar() throws Exception { // Código }
try	Define um bloco para monitorar exceções.	java try { int x = 1 / 0; } catch (Exception e) {}
void	Indica que um método não retorna valor.	java void exibirMensagem() { System.out.println("Olá"); }
while	Define um laço de repetição baseado em uma condição.	java while (x < 5) { System.out.println(x); x++; }

QUESTÃO COMENTADA EM AULA

001. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Na linguagem de programação *Java*, quando uma classe é abstrata:

- a) não se pode realizar a sobrecarga de construtores.
- b) o modificador de acesso deve ser o *protected*.
- c) não pode ser instanciada diretamente.
- d) ela não pode ser herdada.
- e) todos os métodos devem ser abstratos.

EXERCÍCIOS

002. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) O estado de uma classe na linguagem de programação Java é determinado:

- a) pelo seu modificador de acesso.
- b) pelo valor armazenado nos atributos em um determinado momento, durante a execução de um programa.
- c) pela sobrecarga de construtores.
- d) pela sobrescrita de métodos.
- e) pela herança de um objeto polimórfico.

003. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre os construtores na linguagem de programação Java, é correto afirmar que:

- a) deve ter parâmetros e o nome do construtor deve coincidir com o nome da classe.
- b) pode ter retorno e o nome do construtor não precisa coincidir com o nome da classe.
- c) pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.
- d) não pode ter retorno e o nome do construtor deve coincidir com o nome da classe.
- e) não pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.

004. (IV-UFG/CÂMARA DE MORRINHOS-GO/ANALISTA DE TI/2025) A orientação a objetos possui uma série de características próprias que favorecem o desenvolvimento de software. Entre elas, a característica que permite que um método de uma classe derivada tenha a mesma assinatura que um método da classe base, mas com comportamentos diferentes é:

- a) o encapsulamento.
- b) a herança.
- c) a interface.
- d) o polimorfismo.
- e) a mensagem.

005. (UNESPAR/UNESPAR/AGENTE UNIVERSITÁRIO/TÉCNICO EM INFORMÁTICA/2025) Qual das opções NÃO se aplica à programação orientada a objetos em Java?

- a) Encapsulamento.
- b) Herança.
- c) Polimorfismo.
- d) Referências diretas à memória como em C.
- e) Abstração.

006. (FCC/TRT 15ª REGIÃO/ANALISTA JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em Java.

```
class BancoDeDados {
    void conectar(String url) throws Exception {
        if (url == null || url.isEmpty()) {
            throw new Exception("URL inválida.");
        }
        System.out.println("Conectado ao banco: " + url);
    }
}

class ConsultaProcesso extends BancoDeDados {
    @Override
    void conectar(String url) throws Exception {
        System.out.println("Conexão validada.");
            I    
    }
}
```

Com base nos conceitos de herança e sobrescrita e sabendo que o código será executado em condições ideais, a lacuna I deve ser corretamente preenchida com:

- a) super.conectar("");
- b) super.conectar (url);
- c) this.conectar("jdbc:padrao");
- d) system.out.printin("Conexão concluída.");
- e) throw new Exception("Conexão não implementada.");

007. (FADESP/UNIFESSPA/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Na linguagem de programação Java, uma classe pode:

- a) herdar mais de uma classe diretamente. Quando este for o caso, as classes herdadas devem ser públicas.
- b) ter mais de um construtor. Quando este for o caso, a técnica de sobrecarga precisa ser utilizada.
- c) herdar mais de uma classe diretamente. Quando este for o caso, as classes herdadas devem ser protected.
- d) ter mais de um construtor. Quando este for o caso, a técnica de sobrescrita precisa ser utilizada.
- e) herdar mais de uma classe diretamente. Quando este for o caso, as classes devem estar no mesmo pacote.

008. (UFV/UFV-MG/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código escrito na linguagem Java, versão 8:

```
package br.ufv.concurso;

public final class Conexao {

    private static volatile Conexao con;

    private Conexao() {

    }

    public static Conexao getInstance() {
        Conexao conAux = con;
        if (conAux != null) {
            return conAux;
        }
        synchronized(Conexao.class) {
            if (con == null) {
                con = new Conexao();
            }
            return con;
        }
    }
}
```

Assinale a alternativa que apresenta CORRETAMENTE o nome do padrão de projeto que foi implementado na classe Conexao:

- a) Builder
- b) Singleton
- c) Lazy Connection
- d) Synchronized Factory

009. (IF-ES/IF-ES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Na linguagem Java, é uma diretiva utilizada para designar os membros de uma superclasse que podem ser acessados por outros membros dessa superclasse, por membros de subclasses e por membros de outras classes no mesmo pacote:

- a) *volatile*.
- b) *finally*.
- c) *continue*.
- d) *protected*.
- e) *instanceof*.

010. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA INDUSTRIAL DE HEMODERIVADOS E BIOTECNOLOGIA/TECNOLOGIA DA INFORMAÇÃO E OPERAÇÃO/2025) As linguagens de programação são essenciais para o desenvolvimento de sistemas na Hemobrás, especialmente na integração de sistemas industriais. Algumas linguagens são mais utilizadas em ambientes

corporativos e industriais como, por exemplo, a linguagem de programação *Java*. Sobre as características da linguagem do *Java*, assinale a afirmativa correta:

- a) Não possui suporte a *multithreading*, ou seja, execução simultânea de processos.
- b) É uma linguagem compilada que gera código de máquina diretamente executável.
- c) Suporta programação orientada a objetos, permitindo a criação de classes e objetos.
- d) Não é adequada para desenvolvimento de aplicações *web*, sendo recomendada apenas para sistemas de *desktop*.

011. (FUNDAÇÃO CETAP/BANPARÁ/TÉCNICO EM INFORMÁTICA/DESENVOLVIMENTO DE SISTEMAS E ACOMPANHAMENTO DE PROJETOS) Considere os conceitos de Programação Orientada a Objetos (POO) e o seguinte trecho de código em Java e analise as afirmativas subsequentes sobre programação orientada a objetos usando Java.

```
class Veiculo {
    private String marca;
    private String modelo;

    public Veiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    public String descricao() {
        return marca + " " + modelo;
    }
}

class Carro extends Veiculo {
    private int portas;

    public Carro(String marca, String modelo, int portas) {
        super(marca, modelo);
        this.portas = portas;
    }

    @Override
    public String descricao() {
        return super.descricao() + ", " + portas + " portas";
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Carro meuCarro = new Carro("Volkswagen", "Fusca", 2);
        System.out.println(meuCarro.descricao());
    }
}
```

(Código adaptado de código disponível em: <https://www.devmedia.com.br/construtores-em-java-primeiros-passos/28618>)

I – O método descrição da classe Carro sobrescreve o método da classe Veiculo.

II – A classe Veiculo deriva diretamente da superclasse Javamaster.

III – O método descrição da classe Carro chama o método —“descricao da classe Veiculo usando super.descricao(), o que é permitido e comum em POO para estender comportamentos.

Qual(is) afirmativa(s) está(ão) correta(s)?

- a) Todas as três afirmativas estão corretas.
- b) Somente as afirmativas I e III.
- c) Somente as afirmativas II e III.
- d) Somente as afirmativas I e II.
- e) Somente a afirmativa I.

012. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) Uma empresa de TI está desenvolvendo um sistema de monitoramento ambiental para a prefeitura municipal. O sistema precisa modelar diferentes tipos de sensores ambientais, como sensores de temperatura e umidade, seguindo os princípios da Programação Orientada a Objetos (POO). Os desenvolvedores iniciam a modelagem com o seguinte código em Java:

```
public class Sensor { protected String nome;
public Sensor(String nome) { this.nome = nome; }
public void medir() { System.out.println("Medição genérica realizada."); } } public class
SensorTemperatura _____ Sensor { public void medir() { System.out.println("Medição de
temperatura realizada."); } }
```

Assinale a alternativa que completa corretamente a lacuna no código, para que a classe SensorTemperatura herde corretamente o comportamento da classe Sensor:

- a) *public class SensorTemperatura extends Sensor*
- b) *public class SensorTemperatura extends Object*
- c) *public class SensorTemperatura inherits Sensor*
- d) *public class SensorTemperatura implements Sensor*

013. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) Um desenvolvedor precisa revisar um código em Java e verificar se a implementação dos princípios da Programação Orientada a Objetos (POO) está correta. Para isso, ele faz uma análise dos conceitos fundamentais de POO e suas definições. Associe cada conceito da POO à sua respectiva descrição:

1. Herança.

2. Polimorfismo.

3. Encapsulamento.

4. Abstração.

- () Foca em identificar e expor apenas os aspectos essenciais de um objeto ou sistema, simplificando a complexidade ao ocultar detalhes desnecessários para o usuário final.
- () Permite a criação de classes que compartilham características comuns sem duplicação de código.
- () Permite que métodos com o mesmo nome tenham diferentes implementações em classes distintas.
- () Envolve a organização dos dados e métodos relacionados dentro de uma única unidade ou classe, controlando o acesso aos seus componentes internos através de modificadores de acesso, como `private` ou `protected`, para proteger a integridade e a consistência dos dados.

A sequência está correta em:

a) 1, 2, 3, 4.

b) 2, 1, 4, 3.

c) 4, 1, 2, 3.

d) 4, 3, 2, 1.

014. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT/ÁREA INFORMAÇÃO E COMUNICAÇÃO I/2025) Um programador está desenvolvendo um sistema utilizando o paradigma de programação orientada a objetos para um posto de combustível. Neste contexto, considere a definição de classe a seguir utilizando Linguagem Java.

```
class BombaDeCombustivel{ int id; private float qtde; private float precoTotal; public static  
precoPorLitro = 6.10;  
public void setQtde(float qtde){ this.qtde = qtde; this.precoTotal = qtde*this.precoPorLitro; } }
```

Sobre o(os) atributo(s) da classe `BombaDeCombustivel`, é correto afirmar que:

a) `id` e `precoPorLitro` têm visibilidade pública, sendo que `precoPorLitro` é um atributo de classe.

b) `id`, `qtde` e `precoTotal` são visíveis em classes que herdem de `BombaDeCombustivel`.

- c) `precoPorLitro` é uma constante, e seu valor não pode ser alterado em tempo de execução.
- d) `precoPorLitro` é um atributo de classe, portanto seu valor é compartilhado por todas as instâncias desta classe.

015. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Na linguagem de programação *Java*, quando uma classe é abstrata:

- a) não se pode realizar a sobrecarga de construtores.
- b) o modificador de acesso deve ser *protected*.
- c) não pode ser instanciada diretamente.
- d) ela não pode ser herdada.
- e) todos os métodos devem ser abstratos.

016. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Sobre os construtores na linguagem de programação *Java*, é correto afirmar que:

- a) deve ter parâmetros e o nome do construtor deve coincidir com o nome da classe.
- b) pode ter retorno e o nome do construtor não precisa coincidir com o nome da classe.
- c) pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.
- d) não pode ter retorno e o nome do construtor deve coincidir com o nome da classe.
- e) não pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.

017. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Considerando o seguinte código na linguagem de programação *Java*,

```
class Main {
    public static void main(String[] args) {
        double x = 10.5, y = 12.8;
        double z = run(x, y);
        System.out.println(z);
    }
    // inserir código
}
```

informe o código que deve substituir o comentário para que o programa gere a saída 23.0:

- a) `static double run(double x, double y) {return Math.floor(x+y);}`
- b) `public double run(double x, double y) {return Math.ceil(x+y);}`

- c) `static int run(int x, int y) {return Math.floor(x+y)}`
- d) `public double run(double x, double y) {return Math.floor(x+y);}`
- e) `static double run(double x, double y) {return Math.ceil(x+y);}`

018. (FCC/TRT 15ª REGIÃO/TÉCNICO JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) O trecho de código Java (executado em condições ideais, com as bibliotecas adequadas importadas) que não apresenta erros nem exceções é:

- a) `int [] numeros = new int [3]; numeros [3] = 10;`
- b) `string texto = null; System.out.println(texto.toUpperCase());`
- c) `for (int i = 0; i < 10; i++) int x = i * 2; System.out.println(x);`
- d) `Map<String, Integer> mapa = new HashMap<>(); mapa.put ("chave", 42); System.out.println(mapa.getOrDefault ("outraChave", 0) + mapa.get ("chave"));`
- e) `List<Integer> lista = new ArrayList<>(); lista.add("texto");`

019. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

Em Java, os arrays são tipos de dados dinâmicos cujos tamanhos podem ser alterados em tempo de execução.

020. (FCC/TRT 15ª REGIÃO/TÉCNICO JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em Java, utilizando JUnit e Mockito para testar o comportamento de uma classe de serviço.

```
public class PedidoService {
    private final PedidoRepository pedidoRepository;

    public PedidoService(PedidoRepository pedidoRepository) {
        this.pedidoRepository = pedidoRepository;
    }

    public String processarPedido(String pedidoId) {
        Optional<Pedido> pedido = pedidoRepository.buscarPorId(pedidoId);
        if (pedido.isPresent() && pedido.get().isValido()) {
            return "Pedido processado com sucesso";
        } else {
            return "Falha ao processar pedido";
        }
    }
}

@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {
    @Mock
    private PedidoRepository pedidoRepository;

    @InjectMocks
    private PedidoService pedidoService;
```

```
@Test
void testeProcessarPedido_Sucesso() {
    Pedido pedidoMock = Mockito.mock(Pedido.class);
    Mockito.when(pedidoMock.isValido()).thenReturn(false);
    Mockito.when(pedidoRepository.buscarPorId("123")).thenReturn(Optional.of(pedidoMock));
    String resultado = pedidoService.processarPedido("123");
    Assertions.assertEquals("Pedido processado com sucesso", resultado);
}
```

Ao se executar o código em condições ideais, o teste:

- a) falhará porque o método `buscarPorId` do `pedidoRepository` não foi chamado. É necessário usar o `verify` do Mockito para validar a interação.
- b) falhará porque o mock de pedido foi configurado para retornar *false* no método `isValido`, mas o caso de sucesso exige que ele retorne *true*.
- c) está correto, mas o código da classe `PedidoService` está com erro na condição de validação, pois não considera o caso em que `isValido` retorne *false*.
- d) falhará porque a *string* de comparação no `Assertions.assertEquals` está escrita incorretamente. A frase "Pedido processado com sucesso" deveria ser alterada para "Falha ao processar pedido".
- e) falhará porque o método `buscarPorId` do `pedidoRepository` não foi corretamente codificado para retornar um valor válido. É necessário corrigir o mock.

021. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

O ciclo de desenvolvimento em PHP é mais lento que em Java, devido à necessidade de compilar o código a cada alteração antes de executá-lo.

022. (UFV/UFV-MG/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre as interfaces em Java, é INCORRETO afirmar que:

- a) podem ser usadas na marcação de classes, produzindo a noção de relacionamento do tipo "é um".
- b) permitem compartilhar métodos e constantes comuns entre classes não relacionadas hierarquicamente.
- c) podem conter constantes, assinaturas de métodos, métodos estáticos e ser instanciadas diretamente por meio do construtor.
- d) funcionam como contratos de implementação para um componente, permitindo a implementação de funcionalidades por times diferentes.

023. (IBFC/UBFC/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Um computador é uma máquina que processa dados. Ao escrevermos um programa, estamos, na verdade,

realizando essa tarefa com os dados que o usuário entrega para a máquina. Por exemplo, quando queremos que um programa realize o cálculo da média entre duas notas, é preciso que, de alguma forma, o programa “receba” essas duas notas. Assinale a alternativa que apresenta qual operador é comumente utilizado em linguagens baseadas em C, como a linguagem Java por exemplo, para sinalizar o recebimento de um valor para uma variável.

- a) =
- b) <=
- c) <
- d) :=:
- e) =<

024. (CESPE/CEBRASPE/EMBRAPA/ANALISTA/ÁREA GESTÃO DA INFORMAÇÃO/2025) Julgue o próximo item, a respeito de arquitetura de sistemas de computação, sistemas de informação e linguagens de programação.

Em Java, a palavra-chave `static` é utilizada para declarar variáveis locais dentro de um método, para que elas mantenham seu valor entre diferentes chamadas ao método.

025. (CESPE/CEBRASPE/EMBRAPA/ANALISTA/ÁREA GESTÃO DA INFORMAÇÃO/2025) Julgue o seguinte item, referente à programação e estruturas de dados.

O código a seguir, em Java, retornará Tecnológica como resultado de sua execução.

```
import java.util.ArrayList;
```

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> embrapa = new  
        ArrayList<String>();  
        embrapa.add("Ageitec");  
        embrapa.add("Agência");  
        embrapa.add("Embrapa");  
        embrapa.add("de Informação");  
        embrapa.add("Tecnológica");  
        int a=1;  
        for (int i = 0; i < embrapa.size()-2; ++i)  
        {  
            a = a+2;  
        }  
        System.out.println(embrapa.get(a-3));  
    }  
}
```

026. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

A compilação de código em linguagens como Java resulta em bytecode que é interpretado pela JVM, enquanto o PHP é interpretado diretamente sem a necessidade de uma fase de compilação, o que pode levar a um desempenho menor, mas, por outro lado, teremos mais facilidades de depuração.

027. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

Em Java, os tipos de dados primitivos são passados por referência, permitindo que suas mudanças dentro de um método afetem diretamente a variável original.

028. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

A administração de memória em Java é automatizada pelo sistema de garbage collection, que identifica e libera a memória ocupada por objetos que não são mais referenciados, evitando-se vazamentos de memória.

029. (FGV/PC-MG/PERITO CRIMINAL/2025) Considere as características fundamentais da linguagem Java e seus recursos no desenvolvimento de aplicações. Com base nos conceitos básicos de orientação a objetos, em relação ao uso de classes e objetos no Java, assinale a afirmativa CORRETA:

- a) Em Java, uma classe pode ser instanciada apenas se tiver um construtor público ou um construtor com especificadores de acesso adequados.
- b) O modificador public impede que um método ou variável de instância seja acessado fora da sua própria classe.
- c) Uma classe semiabstrata não pode ser instanciada diretamente, mas pode conter métodos implementados e não implementados.
- d) O conceito de polimorfismo permite que um objeto se comporte de forma diferente dependendo do tipo da referência usada para acessá-lo.
- e) O uso da palavra-chave super é utilizado para acessar membros da classe pai em classes que herdam de outra classe.

030. (FCC/PREFEITURA DE SÃO PAULO-SP/ANALISTA DE PLANEJAMENTO E DESENVOLVIMENTO ORGANIZACIONAL TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO/2025) Uma prefeitura está desenvolvendo um sistema integrado de gestão pública que centraliza diversas funções administrativas. Para garantir a consistência dos logs de sistema, foi decidido implementar o padrão *Singleton* para a classe de gerenciamento de logs. Um dos desenvolvedores sugeriu a seguinte implementação em Java:

```
public class LogManager {
    private static LogManager instance;
    private LogManager() {}

    public static LogManager getInstance() {
        if (instance == null) {
            instance = new LogManager();
        }
        return instance;
    }
}
```

Em condições ideais, para garantir que apenas um thread possa executar o método `getInstance` por vez, evitando a criação de múltiplas instâncias em um ambiente *multithread*, e considerando que o desempenho não é a principal prioridade, é adequado:

- a) adicionar o modificador *synchronized* ao método *getInstance* para torná-lo *thread-safe*.
- b) declarar o atributo *instance* como *volatile* para garantir a consistência em ambientes *multithread*.
- c) implementar a inicialização precoce (*lazy initialization*), instanciando *instance* imediatamente após a declaração.
- d) redefinir o construtor como público, garantindo assim que apenas uma instância seja criada a partir de chamadas externas.
- e) adicionar um bloco *try-catch* ao método *getInstance* para lidar com a exceção *MultithreadException*.

031. (FUNDATEC/IF-AM/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre a declaração 'import' na linguagem de programação Java, como, por exemplo "import java.util.Scanner;" analise as assertivas abaixo:

- I – É uma declaração que auxilia o compilador a localizar as classes utilizadas no programa.
- II – Indica que o programa faz uso de uma classe pré-definida do pacote Java.
- III – O compilador não garante que o uso da classe será sempre correto e adequado ao contexto, não sendo necessário nenhum teste adicional.

Quais estão corretas?

- a) Apenas I.
- b) Apenas III.
- c) Apenas I e II.
- d) Apenas II e III.
- e) I, II e III.

032. (CESGRANRIO/UNEMAT/ANALISTA DE SISTEMAS/2024) No processo de geração de código executável em Java, a etapa crucial que envolve a transformação do código-fonte Java em bytecode e o componente responsável por executar esse bytecode são, respectivamente:

- a) bytecode e JIT Compiler
- b) compilação e JRE
- c) compilação e JVM
- d) interpretação e JRE
- e) intermediário e Bytecode Engine

033. (UEM/UEM/ANALISTA DE INFORMÁTICA I/2025) Qual das alternativas apresenta uma instrução válida em Java padrão, sem necessidade de bibliotecas externas, e que imprime uma mensagem no console?

- a) print("Olá, mundo!")
- b) printf("Olá, mundo!");
- c) echo "Olá, mundo!";
- d) System.out.println("Olá, mundo!");
- e) console.log("Olá, mundo!");

034. (QUADRIX/CFP/TÉCNICO ADMINISTRATIVO/SUORTE TÉCNICO EM INFORMÁTICA/2024) Quanto às linguagens de programação Java e PHP, julgue o item a seguir.

O laço de repetição do-while, na linguagem Java, avalia primeiro a expressão condicional e, em seguida, executa o comando.

035. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA INDUSTRIAL DE HEMODERIVADOS E BIOTECNOLOGIA/TECNOLOGIA DA INFORMAÇÃO E OPERAÇÃO/2025) Estruturas de controle de fluxo são fundamentais em *Java* porque permitem que o programador controle o comportamento e a lógica do programa. Em *Java*, qual é a saída do seguinte código que utiliza um *loop* aninhado?

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.print(i * j + " ");
            }
            System.out.println();
        }
    }
}
```

- a) 1 4 9.
- b) 1 2 4 6.
- c) 1 2 2 4 3 6.
- d) Nenhuma saída.

036. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Em Java, a recursão é uma técnica comum, na qual uma função pode se chamar repetidamente até atingir uma condição de parada, sendo utilizada em problemas como pesquisa binária e algoritmos de divisão e conquista.

037. (IMPARH/CGM DE FORTALEZA-CE/AUDITOR DE CONTROLE INTERNO/2025) Sobre os conceitos relacionados à orientação a objetos na linguagem de programação Java, marque a opção CORRETA.

- a) Java permite herança múltipla de classes diretamente.
- b) Todos os métodos de uma classe abstrata devem ser abstratos.
- c) Classes em Java podem implementar várias interfaces.
- d) Uma classe com métodos e variáveis públicas não pode ser encapsulada.

038. (CESPE/CEBRASPE/TRT 10ª REGIÃO/ANALISTA JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Acerca de desenvolvimento de sistemas, julgue o próximo item: Em Java, uma das maneiras de se implementar o polimorfismo é a partir de interfaces que permitem o tratamento de objetos de classes diferentes de forma uniforme.

039. (IBAM/PRODESAN-SP/ANALISTA DE SUPORTE/2025) Os comandos de entrada e saída permitem a interação entre o programa e o usuário, sendo responsáveis pela leitura de dados e exibição de resultados. Acerca desse tema, marque (V), para as afirmativas verdadeiras, e (F), para as falsas.

- () Em linguagens como C, a função scanf permite a leitura de múltiplos valores de entrada, desde que sejam especificados os tipos correspondentes no formato de entrada.
- () A função print em Python é utilizada para exibir dados na saída padrão, sendo possível utilizar múltiplos argumentos separados por vírgulas para concatenar textos e valores.
- () A função cin em C++ lê dados de entrada diretamente do teclado, exigindo que o tipo da variável seja especificado na chamada da função.

- () Em linguagens como Java, o método `System.out.println` é utilizado para exibir dados na saída padrão, adicionando automaticamente uma quebra de linha ao final da impressão.

A sequência está correta em:

- a) V - V - V - V.
- b) V - V - F - V.
- c) V - F - F - F.
- d) F - F - V - V.

040. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

No Java, os subprogramas são definidos por meio de métodos, que podem ser classificados como funções (que retornam um valor) ou procedimentos (que não retornam valor), sendo ambos componentes essenciais para a modularização do código.

041. (FUNDATEC/PREFEITURA DE TANGARÁ DA SERRA/ANALISTA DE SISTEMAS/2025) Analise o trecho de código abaixo, escrito na linguagem de programação Java, e assinale a alternativa que corresponde a uma forma adequada de criar um objeto a partir da classe `Filme`.

```
class Filme {
    String titulo;

    String getTitulo() {
        return titulo;
    }

    void setTitulo(String titulo) {
        this.titulo = titulo;
    }
}
```

- a) `new f = Filme();`
- b) `Filme f = new Filme();`
- c) `(Filme) f = Filme;`
- d) `f = new Filme;`

042. (IDESG/PREFEITURA DE CARIACICA-ES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SISTEMA DA INFORMAÇÃO/2025) Na linguagem de programação Java, pode-se estabelecer

contratos que definem um conjunto de métodos (somente assinaturas dos métodos) que uma classe deve implementar. Este contrato é realizado para especificar um comportamento que várias classes podem compartilhar, sem forçar uma relação de herança direta entre elas. O trecho refere-se à(ao):

- a) Interface.
- b) Encapsulamento.
- c) Método abstrato.
- d) Classe Abstrata.

043. (IDESG/PREFEITURA DE CARIACICA-ES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SISTEMA DA INFORMAÇÃO/2025) Considerando a linguagem de programação Java, este mecanismo consiste em um tipo especial de classe que define um conjunto fixo de constantes (ou valores imutáveis). Ele é usado quando se tem a intenção de representar um grupo de valores relacionados e constantes de forma mais legível e segura no código, viabilizando uma leitura de código mais prática e limpa. Desta forma, qual mecanismo da linguagem de programação Java está sendo descrito?

- a) Atributo privado.
- b) Enum.
- c) Modificadores de acesso.
- d) Constante.

044. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Java não suporta tipos de dados primitivos, sendo obrigado a utilizar apenas objetos para representar valores como inteiros, caracteres e booleanos.

045. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Em Java, as variáveis de tipo primitivo não consomem memória, pois são apenas referências a objetos armazenados na heap.

046. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e

por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

O Java oferece uma forte tipagem estática, o que significa que os tipos de dados de variáveis devem ser explicitamente definidos em tempo de compilação, garantindo maior segurança e detecção precoce de erros.

047. (AVANÇA SP/UNITAU/PROGRAMADOR PLENO/2025) Em Java, qual palavra-chave é usada para definir uma classe que não pode ser estendida?

- a) static
- b) final
- c) private
- d) protected
- e) abstract

048. (UFRR/UFRR/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Python e Java são linguagens de programação muito populares e amplamente utilizadas. Embora ambas sejam linguagens de alto nível, ambas possuem diferenças e semelhanças em alguns aspectos. Analise as alternativas abaixo e encontre a INCORRETA:

- a) Java possui uma sintaxe mais verbosa, enquanto Python possui uma sintaxe mais simples e legível.
- b) Python é conhecida por ser uma linguagem com tipagem dinâmica, ao passo que Java é reconhecida por sua tipagem estática.
- c) Java possui Garbage collector, porém o Python não tem.
- d) Spring é um framework Java e Flask um framework Python.
- e) Java não permite herança múltipla, já o Python permite.

049. (IF-ES/IF-ES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Uma escola deseja implementar um sistema para gerenciar suas listas de alunos e professores, utilizando um aplicativo Java. Como parte do processo de desenvolvimento, o Técnico de TI precisa compilar o código-fonte do aplicativo e garantir que ele seja executado em diferentes sistemas operacionais, sem necessidade de recompilação.

Qual das alternativas descreve CORRETAMENTE o motivo pelo qual os programas Java podem ser executados em várias plataformas?

- a) O compilador Java converte o código-fonte em bytecodes, que são independentes de plataforma e executados pela Java Virtual Machine (JVM).
- b) O código-fonte Java é diretamente interpretado pelo sistema operacional de cada plataforma.

- c) O compilador Java converte o código-fonte em instruções específicas de hardware para cada sistema operacional.
- d) A JVM compila o código-fonte Java em tempo de execução, gerando um arquivo executável para cada sistema operacional.
- e) O ambiente de desenvolvimento integrado (IDE) utilizado para desenvolver o programa garante sua compatibilidade em todas as plataformas.

050. (INÉDITA/2025) Qual das seguintes afirmações sobre métodos estáticos em Java é verdadeira?

- a) Podem acessar diretamente variáveis de instância.
- b) Podem ser chamados sem a criação de um objeto da classe.
- c) Devem ser declarados dentro de um método principal.
- d) Precisam de uma instância de classe para serem invocados.

GABARITO

- | | |
|-------|-------|
| 1. c | 35. c |
| 2. b | 36. C |
| 3. d | 37. c |
| 4. d | 38. C |
| 5. d | 39. b |
| 6. b | 40. C |
| 7. b | 41. b |
| 8. b | 42. a |
| 9. d | 43. b |
| 10. c | 44. E |
| 11. b | 45. E |
| 12. a | 46. C |
| 13. c | 47. b |
| 14. d | 48. c |
| 15. c | 49. a |
| 16. d | 50. b |
| 17. a | |
| 18. d | |
| 19. E | |
| 20. b | |
| 21. E | |
| 22. c | |
| 23. a | |
| 24. E | |
| 25. C | |
| 26. E | |
| 27. E | |
| 28. C | |
| 29. e | |
| 30. a | |
| 31. c | |
| 32. c | |
| 33. d | |
| 34. E | |

GABARITO COMENTADO

002. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) O estado de uma classe na linguagem de programação Java é determinado:

- a) pelo seu modificador de acesso.
- b) pelo valor armazenado nos atributos em um determinado momento, durante a execução de um programa.
- c) pela sobrecarga de construtores.
- d) pela sobrescrita de métodos.
- e) pela herança de um objeto polimórfico.



- a) Errada. O modificador de acesso (public, private, protected, default) define a visibilidade dos membros de uma classe, mas não o seu estado. O estado é dinâmico e depende dos dados.
- b) Certa. O estado de um objeto (instância de uma classe) é definido pelos valores atuais de seus atributos (variáveis de instância) em um dado momento durante a execução do programa. Cada objeto pode ter um estado diferente.
- c) Errada. A sobrecarga de construtores permite ter múltiplos construtores com diferentes listas de parâmetros, mas isso se refere à forma como um objeto é inicializado, não ao seu estado em si.
- d) Errada. A sobrescrita de métodos (method overriding) é um conceito de polimorfismo que permite a uma subclasse fornecer uma implementação específica para um método que já é fornecido por uma de suas superclasses. Isso afeta o comportamento, não o estado.
- e) Errada. A herança e o polimorfismo são pilares da POO que permitem a reutilização de código e a flexibilidade, mas não definem diretamente o estado de uma classe ou de suas instâncias.

Letra b.

003. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre os construtores na linguagem de programação Java, é correto afirmar que:

- a) deve ter parâmetros e o nome do construtor deve coincidir com o nome da classe.
- b) pode ter retorno e o nome do construtor não precisa coincidir com o nome da classe.
- c) pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.
- d) não pode ter retorno e o nome do construtor deve coincidir com o nome da classe.
- e) não pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.



- a) Errada. Um construtor não deve ter parâmetros; ele pode ter um construtor padrão (sem parâmetros). No entanto, o nome do construtor deve coincidir com o nome da classe.
- b) Errada. Construtores em Java não podem ter um tipo de retorno (nem mesmo void), e o nome do construtor deve coincidir com o nome da classe.
- c) Errada. Construtores podem ter parâmetros, mas o nome do construtor deve coincidir com o nome da classe.
- d) Certa. Esta é uma característica fundamental dos construtores em Java: eles não possuem tipo de retorno (nem void) e seu nome deve ser exatamente o mesmo da classe que estão construindo.
- e) Errada. Construtores podem ter parâmetros (para inicializar o estado do objeto), e o nome do construtor deve coincidir com o nome da classe.

Letra d.

004. (IV-UFG/CÂMARA DE MORRINHOS–GO/ANALISTA DE TI/2025) A orientação a objetos possui uma série de características próprias que favorecem o desenvolvimento de software. Entre elas, a característica que permite que um método de uma classe derivada tenha a mesma assinatura que um método da classe base, mas com comportamentos diferentes é:

- a) o encapsulamento.
- b) a herança.
- c) a interface.
- d) o polimorfismo.
- e) a mensagem.



- a) Errada. Encapsulamento é o princípio de agrupar dados (atributos) e métodos que operam sobre esses dados em uma única unidade (classe), controlando o acesso externo a eles. Não se refere à mudança de comportamento de métodos herdados.
- b) Errada. Herança é o mecanismo que permite que uma classe (subclasse) adquira as características (atributos e métodos) de outra classe (superclasse). Embora seja um pré-requisito para a sobrescrita, a herança por si só não descreve a mudança de comportamento.
- c) Errada. Interface em Java define um contrato de métodos que uma classe deve implementar, mas não é a característica que permite a um método herdado ter comportamento diferente.
- d) Certa. Polimorfismo (especificamente o polimorfismo de sobrescrita ou overriding) é a característica que permite que métodos com a mesma assinatura (nome e parâmetros) em classes diferentes (uma superclasse e suas subclasses) tenham implementações distintas, resultando em comportamentos diferentes.

e) Errada. Mensagem é o termo usado para descrever a invocação de um método em um objeto, mas não é uma característica da POO que define a mudança de comportamento de métodos herdados.

Letra d.

005. (UNESPAR/UNESPAR/AGENTE UNIVERSITÁRIO/TÉCNICO EM INFORMÁTICA/2025) Qual das opções NÃO se aplica à programação orientada a objetos em Java?

- a) Encapsulamento.
- b) Herança.
- c) Polimorfismo.
- d) Referências diretas à memória como em C.
- e) Abstração.



- a) Errada. Encapsulamento é um dos pilares da POO e é amplamente utilizado em Java para proteger os dados e controlar o acesso a eles.
- b) Errada. Herança é um conceito fundamental da POO em Java, permitindo a reutilização de código e a criação de hierarquias de classes.
- c) Errada. Polimorfismo é outro pilar da POO em Java, que permite que objetos de diferentes classes sejam tratados de forma uniforme através de uma interface comum.
- d) Certa. Java é uma linguagem de alto nível que gerencia a memória automaticamente através do Garbage Collector, e não permite o acesso direto à memória por meio de ponteiros ou referências diretas como em linguagens de baixo nível como C ou C++. Isso é uma característica de segurança e simplificação da linguagem.
- e) Errada. Abstração é um pilar da POO em Java, que permite focar nos aspectos essenciais de um objeto, ocultando os detalhes de implementação. Classes abstratas e interfaces são exemplos de abstração em Java.

Letra d.

006. (FCC/TRT 15ª REGIÃO/ANALISTA JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em Java.

```
class BancoDeDados {
    void conectar(String url) throws Exception {
        if (url == null || url.isEmpty()) {
            throw new Exception("URL inválida.");
        }
        System.out.println("Conectado ao banco: " + url);
    }
}

class ConsultaProcesso extends BancoDeDados {
    @Override
    void conectar(String url) throws Exception {
        System.out.println("Conexão validada.");
        I
    }
}
```

Com base nos conceitos de herança e sobrescrita e sabendo que o código será executado em condições ideais, a lacuna I deve ser corretamente preenchida com:

- a) `super.conectar("");`
- b) `super.conectar(url);`
- c) `this.conectar("jdbc:padrao");`
- d) `system.out.printin("Conexão concluída.");`
- e) `throw new Exception("Conexão não implementada.");`



- a) Errada. Embora `super.conectar()` seja a sintaxe correta para chamar o método da superclasse, a alternativa `super.conectar("");` passa uma string vazia, o que pode não ser o parâmetro esperado ou ideal para uma conexão.
- b) Certa. A sintaxe `super.conectar(url);` é a forma padrão e correta de chamar um método conectar da superclasse, passando uma variável url (que se presume estar disponível no escopo) como argumento. Isso é comum quando uma subclasse sobrescreve um método, mas ainda precisa invocar a funcionalidade da superclasse antes ou depois de sua própria lógica.
- c) Errada. `this.conectar("jdbc:padrao");` chamaria o método conectar da própria classe (ou seja, a versão sobrescrita, se houver, ou a versão herdada se não houver sobrescrita). Se o objetivo é chamar a implementação da superclasse, `super` é necessário.
- d) Errada. `system.out.printin("Conexão concluída.");` é uma linha de impressão e não preenche a lacuna de uma chamada de método relacionada à conexão. Além disso, `printin` é um erro de digitação para `println`.

e) Errada. `throw new Exception("Conexão não implementada.");` lançaria uma exceção, indicando que a conexão não foi implementada, o que contradiz a ideia de preencher a lacuna para que o código seja executado "em condições ideais" e complete uma ação de conexão.

Letra b.

007. (FADESP/UNIFESSPA/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Na linguagem de programação Java, uma classe pode:

- a) herdar mais de uma classe diretamente. Quando este for o caso, as classes herdadas devem ser públicas.
- b) ter mais de um construtor. Quando este for o caso, a técnica de sobrecarga precisa ser utilizada.
- c) herdar mais de uma classe diretamente. Quando este for o caso, as classes herdadas devem ser `protected`.
- d) ter mais de um construtor. Quando este for o caso, a técnica de sobrescrita precisa ser utilizada.
- e) herdar mais de uma classe diretamente. Quando este for o caso, as classes devem estar no mesmo pacote.



- a) Errada. Java não permite herança múltipla de classes (uma classe não pode herdar diretamente de mais de uma superclasse). Isso é uma decisão de design para evitar o "problema do diamante".
- b) Certa. Uma classe em Java pode ter múltiplos construtores, desde que cada um tenha uma lista de parâmetros diferente (número, tipo ou ordem dos parâmetros). Isso é conhecido como sobrecarga de construtores (constructor overloading).
- c) Errada. Assim como na alternativa 'a', Java não permite herança múltipla de classes.
- d) Errada. Para ter mais de um construtor, a técnica utilizada é a sobrecarga (overloading), não a sobrescrita (overriding). Sobrescrita se aplica a métodos em hierarquias de herança.
- e) Errada. Novamente, Java não permite herança múltipla de classes. Além disso, classes podem herdar de classes em pacotes diferentes, desde que a superclasse seja acessível (por exemplo, `public`).

Letra b.

008. (UFV/UFV-MG/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código escrito na linguagem Java, versão 8:

```
package br.ufv.concurso;

public final class Conexao {

    private static volatile Conexao con;

    private Conexao() {

    }

    public static Conexao getInstance() {
        Conexao conAux = con;
        if (conAux != null) {
            return conAux;
        }
        synchronized(Conexao.class) {
            if (con == null) {
                con = new Conexao();
            }
            return con;
        }
    }
}
```

Assinale a alternativa que apresenta CORRETAMENTE o nome do padrão de projeto que foi implementado na classe Conexao:

- a) Builder
- b) Singleton
- c) Lazy Connection
- d) Synchronized Factory



- a) Errada. O padrão Builder é usado para construir objetos complexos passo a passo, permitindo a criação de diferentes representações do mesmo objeto. Não é o objetivo principal de uma classe de conexão que geralmente busca uma única instância.
- b) Certa. O padrão Singleton garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a essa instância. Isso é ideal para recursos como conexões de banco de dados, gerenciadores de log ou configurações, onde apenas uma instância é necessária para todo o sistema.
- c) Errada. "Lazy Connection" não é um padrão de projeto GoF (Gang of Four) reconhecido. Pode ser uma técnica de inicialização preguiçosa (lazy initialization) aplicada a uma conexão, mas não um padrão de projeto por si só.
- d) Errada. "Synchronized Factory" não é um padrão de projeto GoF. O padrão Factory Method ou Abstract Factory são usados para criar objetos, mas não necessariamente para garantir uma única instância. A sincronização pode ser usada em um Factory para garantir thread-safety, mas não define o padrão em si.

Letra b.

009. (IF-ES/IF-ES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Na linguagem Java, é uma diretiva utilizada para designar os membros de uma superclasse que podem ser acessados por outros membros dessa superclasse, por membros de subclasses e por membros de outras classes no mesmo pacote:

- a) *volatile*.
- b) *finally*.
- c) *continue*.
- d) *protected*.
- e) *instanceof*.



a) Errada. *volatile* é um modificador para campos que garante que as leituras e escritas de uma variável sejam diretamente da memória principal, e não de caches de CPU, sendo importante em programação concorrente. Não define regras de acesso.

b) Errada. *finally* é uma palavra-chave usada em blocos try-catch-finally para definir um bloco de código que sempre será executado, independentemente de uma exceção ter sido lançada ou não. Não define regras de acesso.

c) Errada. *continue* é uma instrução de controle de fluxo usada dentro de loops para pular a iteração atual e prosseguir para a próxima. Não define regras de acesso.

d) Certa. *protected* é um modificador de acesso em Java. Membros (atributos e métodos) declarados como *protected* são acessíveis:

- Dentro da própria classe;
- Por subclasses (mesmo que estejam em pacotes diferentes);
- Por outras classes no mesmo pacote.

e) Errada. *instanceof* é um operador que verifica se um objeto é uma instância de uma classe específica ou de uma interface. Não define regras de acesso.

Letra d.

010. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA INDUSTRIAL DE HEMODERIVADOS E BIOTECNOLOGIA/TECNOLOGIA DA INFORMAÇÃO E OPERAÇÃO/2025) As linguagens de programação são essenciais para o desenvolvimento de sistemas na Hemobrás, especialmente na integração de sistemas industriais. Algumas linguagens são mais utilizadas em ambientes corporativos e industriais como, por exemplo, a linguagem de programação Java. Sobre as características da linguagem do Java, assinale a afirmativa correta:

- a) Não possui suporte a *multithreading*, ou seja, execução simultânea de processos.
- b) É uma linguagem compilada que gera código de máquina diretamente executável.

- c) Suporta programação orientada a objetos, permitindo a criação de classes e objetos.
- d) Não é adequada para desenvolvimento de aplicações *web*, sendo recomendada apenas para sistemas de *desktop*.



- a) Errada. Java possui suporte robusto e nativo a multithreading, com classes e APIs dedicadas para a criação e gerenciamento de threads, permitindo a execução simultânea de processos ou partes de um programa.
- b) Errada. Java é uma linguagem compilada, mas não gera código de máquina diretamente executável para uma plataforma específica. Em vez disso, o código-fonte Java é compilado em bytecode, que é um formato intermediário. Este bytecode é então interpretado e executado pela Java Virtual Machine (JVM), o que confere a Java sua característica de portabilidade (“Write Once, Run Anywhere”).
- c) Certa. Java é uma linguagem fundamentalmente orientada a objetos. Ela é construída em torno dos conceitos de classes, objetos, herança, polimorfismo, encapsulamento e abstração, sendo um dos principais paradigmas de programação que suporta.
- d) Errada. Java é extremamente adequada e amplamente utilizada para o desenvolvimento de aplicações web (com frameworks como Spring, Jakarta EE etc.), aplicações corporativas, mobile (Android), sistemas embarcados e, claro, sistemas de desktop. Afirmar que é recomendada “apenas para sistemas de desktop” é falso.

Letra c.

011. (FUNDAÇÃO CETAP/BANPARÁ/TÉCNICO EM INFORMÁTICA/DESENVOLVIMENTO DE SISTEMAS E ACOMPANHAMENTO DE PROJETOS) Considere os conceitos de Programação Orientada a Objetos (POO) e o seguinte trecho de código em Java e analise as afirmativas subsequentes sobre programação orientada a objetos usando Java.

```
class Veiculo {
    private String marca;
    private String modelo;

    public Veiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    public String descricao() {
        return marca + " " + modelo;
    }
}
```

```
class Carro extends Veiculo {
    private int portas;

    public Carro(String marca, String modelo, int portas) {
        super(marca, modelo);
        this.portas = portas;
    }

    @Override
    public String descricao() {
        return super.descricao() + ", " + portas + " portas";
    }
}

public class Main {
    public static void main(String[] args) {
        Carro meuCarro = new Carro("Volkswagen", "Fusca", 2);
        System.out.println(meuCarro.descricao());
    }
}
```

(Código adaptado de código disponível em: <https://www.devmedia.com.br/construtores-em-java-primeiros-passos/28618>)

- I – O método descrição da classe Carro sobrescreve o método da classe Veiculo.
 - II – A classe Veiculo deriva diretamente da superclasse Javamaster.
 - III – O método descrição da classe Carro chama o método —“descricao da classe Veiculo usando super.descricao(), o que é permitido e comum em POO para estender comportamentos.
- Qual(is) afirmativa(s) está(ão) correta(s)?
- a) Todas as três afirmativas estão corretas.
 - b) Somente as afirmativas I e III.
 - c) Somente as afirmativas II e III.
 - d) Somente as afirmativas I e II.
 - e) Somente a afirmativa I.



- I – Certa. Em POO, é uma prática comum que uma subclasse (Carro) sobrescreva um método (descrição) de sua superclasse (Veiculo) para fornecer uma implementação mais específica. Isso é um exemplo de polimorfismo.
- II – Errada. Em Java, todas as classes, se não estenderem explicitamente outra classe, derivam implicitamente da classe java.lang.Object. “Javamaster” não é uma superclasse padrão em Java. Sem o código, não podemos confirmar a existência de uma classe Javamaster

específica, mas a afirmação de que Veiculo deriva diretamente dela como uma superclasse padrão é falsa.

III – Certa. Quando um método é sobrescrito em uma subclasse, é uma prática comum e permitida chamar a implementação do método da superclasse usando a palavra-chave `super` (ex: `super.descricao()`). Isso permite que a subclasse estenda ou complemente o comportamento da superclasse, em vez de substituí-lo completamente.

Conclusão: as afirmativas I e III estão corretas, enquanto a afirmativa II está incorreta.

Letra b.

012. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) Uma empresa de TI está desenvolvendo um sistema de monitoramento ambiental para a prefeitura municipal. O sistema precisa modelar diferentes tipos de sensores ambientais, como sensores de temperatura e umidade, seguindo os princípios da Programação Orientada a Objetos (POO). Os desenvolvedores iniciam a modelagem com o seguinte código em Java:

```
public class Sensor { protected String nome;
public Sensor(String nome) { this.nome = nome; }
public void medir() { System.out.println("Medição genérica realizada."); } }
public class SensorTemperatura _____ Sensor { public void medir() { System.out.println("Medição de
temperatura realizada."); } }
```

Assinale a alternativa que completa corretamente a lacuna no código, para que a classe `SensorTemperatura` herde corretamente o comportamento da classe `Sensor`:

- a) *public class SensorTemperatura extends Sensor*
- b) *public class SensorTemperatura extends Object*
- c) *public class SensorTemperatura inherits Sensor*
- d) *public class SensorTemperatura implements Sensor*



a) Certa. Em Java, a palavra-chave `extends` é usada para indicar que uma classe está herdando de outra classe. Portanto, `public class SensorTemperatura extends Sensor` significa que `SensorTemperatura` é uma subclasse de `Sensor`.

b) Errada. Embora todas as classes em Java herdem implicitamente de `Object` (se não estenderem outra classe), `extends Object` seria redundante e não estabeleceria a herança específica de `Sensor`.

c) Errada. `Inherits` não é uma palavra-chave válida para herança em Java. É usada em algumas outras linguagens, mas não em Java.

d) Errada. implements é a palavra-chave usada para indicar que uma classe está implementando uma interface, não herdando de outra classe.

Letra a.

013. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) Um desenvolvedor precisa revisar um código em Java e verificar se a implementação dos princípios da Programação Orientada a Objetos (POO) está correta. Para isso, ele faz uma análise dos conceitos fundamentais de POO e suas definições. Associe cada conceito da POO à sua respectiva descrição:

1. Herança.

2. Polimorfismo.

3. Encapsulamento.

4. Abstração.

- () Foca em identificar e expor apenas os aspectos essenciais de um objeto ou sistema, simplificando a complexidade ao ocultar detalhes desnecessários para o usuário final.
- () Permite a criação de classes que compartilham características comuns sem duplicação de código.
- () Permite que métodos com o mesmo nome tenham diferentes implementações em classes distintas.
- () Envolve a organização dos dados e métodos relacionados dentro de uma única unidade ou classe, controlando o acesso aos seus componentes internos através de modificadores de acesso, como private ou protected, para proteger a integridade e a consistência dos dados.

A sequência está correta em:

- a) 1, 2, 3, 4.
- b) 2, 1, 4, 3.
- c) 4, 1, 2, 3.
- d) 4, 3, 2, 1.



Vamos associar cada conceito à sua descrição:

(4) Abstração. Foca em identificar e expor apenas os aspectos essenciais de um objeto ou sistema, simplificando a complexidade ao ocultar detalhes desnecessários para o usuário final.

(1) Herança. Permite a criação de classes que compartilham características comuns sem duplicação de código.

(2) Polimorfismo. Permite que métodos com o mesmo nome tenham diferentes implementações em classes distintas.

(3) Encapsulamento. Envolve a organização dos dados e métodos relacionados dentro de uma única unidade ou classe, controlando o acesso aos seus componentes internos através de modificadores de acesso, como `private` ou `protected`, para proteger a integridade e a consistência dos dados.

Portanto, a sequência correta é 4, 1, 2, 3.

Letra c.

014. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT/ÁREA INFORMAÇÃO E COMUNICAÇÃO I/2025) Um programador está desenvolvendo um sistema utilizando o paradigma de programação orientada a objetos para um posto de combustível. Neste contexto, considere a definição de classe a seguir utilizando Linguagem Java.

```
class BombaDeCombustivel{ int id; private float qtde; private float precoTotal; public static precoPorLitro = 6.10;
```

```
public void setQtde(float qtde){ this.qtde = qtde; this.precoTotal = qtde*this.precoPorLitro; }}
```

Sobre o(os) atributo(s) da classe `BombaDeCombustivel`, é correto afirmar que:

- a) `id` e `precoPorLitro` têm visibilidade pública, sendo que `precoPorLitro` é um atributo de classe.
- b) `id`, `qtde` e `precoTotal` são visíveis em classes que herdem de `BombaDeCombustivel`.
- c) `precoPorLitro` é uma constante, e seu valor não pode ser alterado em tempo de execução.
- d) `precoPorLitro` é um atributo de classe, portanto seu valor é compartilhado por todas as instâncias desta classe.



Vamos analisar os atributos da classe `BombaDeCombustivel`:

- `int id;` – Não possui modificador de acesso explícito, o que significa que tem visibilidade `package-private` (acessível apenas dentro do mesmo pacote);
- `private float qtde;` – Possui visibilidade `private`, acessível apenas dentro da própria classe;
- `private float precoTotal;` – Possui visibilidade `private`, acessível apenas dentro da própria classe;
- `public static precoPorLitro = 6.10;` – Possui visibilidade `public` e é `static`. O `static` significa que é um atributo de classe, pertencente à classe `BombaDeCombustivel` e não a instâncias individuais.

a) Errada. `id` tem visibilidade `package-private`, não pública. `precoPorLitro` tem visibilidade pública e é um atributo de classe, mas a primeira parte da afirmação está errada.

- b) Errada. `qtde` e `precoTotal` são `private`, o que significa que não são visíveis nem mesmo para classes que herdem de `BombaDeCombustivel`. `id` seria visível apenas se a classe herdeira estivesse no mesmo pacote.
- c) Errada. `precoPorLitro` é `static` e `public`, mas não é `final`. Para ser uma constante cujo valor não pode ser alterado em tempo de execução, ele precisaria ser declarado como `final` (ex: `public static final float precoPorLitro = 6.10;`). Sem o `final`, seu valor pode ser alterado.
- d) Certa. `precoPorLitro` é declarado com o modificador `static`, o que o torna um atributo de classe. Atributos de classe são compartilhados por todas as instâncias da classe, ou seja, há apenas uma cópia desse atributo para a classe, independentemente de quantas instâncias sejam criadas.

Letra d.

015. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Na linguagem de programação *Java*, quando uma classe é abstrata:

- a) não se pode realizar a sobrecarga de construtores.
- b) o modificador de acesso deve ser o *protected*.
- c) não pode ser instanciada diretamente.
- d) ela não pode ser herdada.
- e) todos os métodos devem ser abstratos.



- a) Errada. Classes abstratas podem ter construtores e podem ter sobrecarga de construtores normalmente. A única limitação é que não podem ser instanciadas diretamente, mas seus construtores são chamados pelas subclasses.
- b) Errada. Não há exigência de que o modificador de uma classe abstrata seja `protected`. Ela pode ser `public`, `protected`, ou sem modificador (`package-private`), conforme necessário. `Protected` é restrito a membros (atributos e métodos), não a classes `top-level`.
- c) Certa. Esta é a característica principal de uma classe abstrata; não pode ser instanciada diretamente. Ela serve como base para outras classes que a estendem.
- d) Errada. Classes abstratas são feitas para serem herdadas. Esse é um dos principais usos da abstração: criar uma estrutura comum que será estendida por subclasses concretas.
- e) Errada. Uma classe abstrata pode conter métodos abstratos ou não. Ela pode ter métodos com implementação completa. Apenas classes que são 100% abstratas (interfaces até o Java 7) tinham essa restrição.

Letra c.

016. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Sobre os construtores na linguagem de programação Java, é correto afirmar que:

- a) deve ter parâmetros e o nome do construtor deve coincidir com o nome da classe.
- b) pode ter retorno e o nome do construtor não precisa coincidir com o nome da classe.
- c) pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.
- d) não pode ter retorno e o nome do construtor deve coincidir com o nome da classe.
- e) não pode ter parâmetros e o nome do construtor não precisa coincidir com o nome da classe.



- a) Errada. Um construtor não deve ter parâmetros; ele pode ter um construtor padrão (sem parâmetros). No entanto, o nome do construtor deve coincidir com o nome da classe.
- b) Errada. Construtores em Java não podem ter um tipo de retorno (nem mesmo void), e o nome do construtor deve coincidir com o nome da classe.
- c) Errada. Construtores podem ter parâmetros, mas o nome do construtor deve coincidir com o nome da classe.
- d) Certa. Esta é uma característica fundamental dos construtores em Java: eles não possuem tipo de retorno (nem void) e seu nome deve ser exatamente o mesmo da classe que estão construindo.
- e) Errada. Construtores podem ter parâmetros (para inicializar o estado do objeto), e o nome do construtor deve coincidir com o nome da classe.

Letra d.

017. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Considerando o seguinte código na linguagem de programação Java,

```
class Main {
    public static void main(String[] args) {
        double x = 10.5, y = 12.8;
        double z = run(x, y);
        System.out.println(z);
    }
    // inserir código
}
```

informe o código que deve substituir o comentário para que o programa gere a saída 23.0:

- a) `static double run(double x, double y) {return Math.floor(x+y);}`
- b) `public double run(double x, double y) {return Math.ceil(x+y);}`
- c) `static int run(int x, int y) {return Math.floor(x+y)}`
- d) `public double run(double x, double y) {return Math.floor(x+y);}`
- e) `static double run(double x, double y) {return Math.ceil(x+y);}`



- a) Certa. O método `Math.floor()` retorna o maior número inteiro (representado como `double`) menor ou igual ao argumento. Se a soma `x+y` for, por exemplo, 23.0, 23.1, 23.5, 23.9, o `Math.floor()` resultará em 23.0. A declaração `static` é comum para métodos utilitários que não dependem do estado de um objeto.
- b) Errada. O método `Math.ceil()` retorna o menor número inteiro (representado como `double`) maior ou igual ao argumento. Se a soma `x+y` for, por exemplo, 22.1, 22.5, 22.9, o `Math.ceil()` resultaria em 23.0. No entanto, se a soma for 23.1, `Math.ceil()` resultaria em 24.0. Sem o contexto dos valores de `x` e `y`, `Math.floor` é uma escolha mais comum para “arredondar para baixo” para um valor exato como 23.0 quando o valor original é 23.alguma_coisa.
- c) Errada. Esta alternativa retorna um `int`, mas a saída esperada é 23.0 (um `double`). Além disso, `Math.floor` sempre retorna um `double`, então haveria um erro de compilação ao tentar atribuir um `double` a um `int` sem um `cast` explícito.
- d) Errada. Esta alternativa é funcionalmente idêntica à letra ‘a’ em termos de cálculo e tipo de retorno. A diferença é o modificador de acesso (`public` vs `static`). No entanto, em um contexto de questão de múltipla escolha, se ambas as opções são válidas para o resultado, a que melhor se encaixa em um cenário de “utilitário” (como `static`) ou a que é considerada a “melhor” resposta sem mais contexto é a preferida. A opção ‘a’ é `static`, o que é comum para funções matemáticas.
- e) Errada. Similar à alternativa ‘b’, `Math.ceil` arredonda para cima. Se a soma `x+y` for 23.1, o resultado seria 24.0, o que não é a saída desejada de 23.0.

Letra a.

018. (FCC/TRT 15ª REGIÃO/TÉCNICO JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) O trecho de código Java (executado em condições ideais, com as bibliotecas adequadas importadas) que não apresenta erros nem exceções é:

- a) `int [] numeros = new int [3]; numeros [3] = 10;`
- b) `string texto = null; System.out.println(texto.toUpperCase());`
- c) `for (int i – 0; i < 10; i++) int x = i * 2; System.out.println(x);`
- d) `Map<String, Integer> mapa = new HashMap<>(); mapa.put (“chave”, 42); System.out.println(mapa.getDefault (“outraChave”, 0) + mapa.get (“chave”));`
- e) `List<Integer> lista = new ArrayList<>(); lista.add(“texto”);`



- a) Errada. Erro/Exceção: `ArrayIndexOutOfBoundsException`. Um array de tamanho 3 tem índices válidos de 0, 1 e 2. Tentar acessar `numeros[3]` está fora dos limites do array.
- b) Errada. Erro/Exceção: `NullPointerException`. A variável `texto` é null. Tentar chamar um método (`toUpperCase()`) em uma referência null resultará em `NullPointerException` em tempo de execução. Além disso, `println` é um erro de digitação para `println`.
- c) Errada. Erro/Exceção: Erro de compilação. A sintaxe `int i – 0;` está incorreta; deveria ser `int i = 0;`. Além disso, `System.out.println(x);` tem um erro de digitação (`println` em vez de `println`). Mesmo que o erro de digitação fosse corrigido, a variável `x` é declarada dentro do escopo do loop `for` e não estaria acessível fora dele, o que causaria um erro de compilação se a linha `System.out.println(x);` estivesse fora do loop. Se a linha de impressão estivesse dentro do loop, o erro de sintaxe `i – 0` ainda impediria a compilação.
- d) Certa. Sem Erros/Exceções: Este código é sintaticamente correto e não gerará exceções em tempo de execução.

`mapa.put("chave", 42);` adiciona um par chave-valor ao mapa.

`mapa.getDefault("outraChave", 0)` tenta obter o valor associado a "outraChave". Como "outraChave" não existe no mapa, ele retorna o valor padrão 0.

`mapa.get("chave")` retorna o valor associado a "chave", que é 42.

A expressão `0 + 42` resulta em 42, que será impresso no console.

- e) Errada. Erro/Exceção: Erro de compilação. `nteger` é um erro de digitação para `Integer`. Mesmo que fosse `Integer`, a linha `lista.add("texto");` causaria um erro de compilação porque a lista é tipada para `Integer` (`List<Integer>`), e você está tentando adicionar uma `String`.

Letra d.

019. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

Em Java, os arrays são tipos de dados dinâmicos cujos tamanhos podem ser alterados em tempo de execução.



Em Java, os arrays são de tamanho fixo. Uma vez que um array é criado com um determinado tamanho, esse tamanho não pode ser alterado. Se você precisar de uma coleção cujo tamanho possa variar dinamicamente, você deve usar classes da API de Coleções, como `ArrayList`, `LinkedList`, etc., que são construídas sobre arrays, mas fornecem uma abstração para gerenciar o redimensionamento interno.

Errado.

020. (FCC/TRT 15ª REGIÃO/TÉCNICO JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em Java, utilizando JUnit e Mockito para testar o comportamento de uma classe de serviço.

```
public class PedidoService {
    private final PedidoRepository pedidoRepository;

    public PedidoService(PedidoRepository pedidoRepository) {
        this.pedidoRepository = pedidoRepository;
    }

    public String processarPedido(String pedidoId) {
        Optional<Pedido> pedido = pedidoRepository.buscarPorId(pedidoId);
        if (pedido.isPresent() && pedido.get().isValido()) {
            return "Pedido processado com sucesso";
        } else {
            return "Falha ao processar pedido";
        }
    }
}

@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {
    @Mock
    private PedidoRepository pedidoRepository;

    @InjectMocks
    private PedidoService pedidoService;

    @Test
    void testeProcessarPedido_Sucesso() {
        Pedido pedidoMock = Mockito.mock(Pedido.class);
        Mockito.when(pedidoMock.isValido()).thenReturn(false);
        Mockito.when(pedidoRepository.buscarPorId("123")).thenReturn(Optional.of(pedidoMock));
        String resultado = pedidoService.processarPedido("123");
        Assertions.assertEquals("Pedido processado com sucesso", resultado);
    }
}
```

Ao se executar o código em condições ideais, o teste:

- falhará porque o método `buscarPorId` do `pedidoRepository` não foi chamado. É necessário usar o `verify` do Mockito para validar a interação.
- falhará porque o mock de `pedido` foi configurado para retornar *false* no método `isValido`, mas o caso de sucesso exige que ele retorne *true*.
- está correto, mas o código da classe `PedidoService` está com erro na condição de validação, pois não considera o caso em que `isValido` retorne *false*.
- falhará porque a *string* de comparação no `Assertions.assertEquals` está escrita incorretamente. A frase "Pedido processado com sucesso" deveria ser alterada para "Falha ao processar pedido".
- falhará porque o método `buscarPorId` do `pedidoRepository` não foi corretamente codificado para retornar um valor válido. É necessário corrigir o mock.



- Errada. Embora seja uma boa prática usar `Mockito.verify()` para garantir que interações específicas ocorreram, a ausência de um `verify` não causa uma falha no teste por si só, a

menos que o teste esteja especificamente verificando a ausência de uma chamada ou que a lógica subsequente dependa dessa chamada para falhar. Se o método não foi chamado e isso é um problema, o teste deveria falhar por uma asserção de resultado, ou por um `verify` explícito.

b) Certa. Se um teste está projetado para validar um “caso de sucesso” (ou seja, espera que o processamento seja bem-sucedido), mas uma dependência mockada (o pedido neste caso) é configurada para retornar `false` para um método de validação (`isValido`), a lógica do serviço (`PedidoService`) provavelmente seguirá um caminho de falha. Isso fará com que as asserções de sucesso do teste falhem, resultando na falha do teste. É uma inconsistência lógica entre a configuração do mock e a expectativa do teste.

c) Errada. Esta alternativa sugere que o teste está correto, mas o código de produção (`PedidoService`) tem um erro. Embora isso seja possível na vida real, a questão pergunta sobre o comportamento do teste. Se o teste está correto, ele deveria falhar se o código de produção estiver errado, ou passar se o código estiver certo. A alternativa ‘b’ descreve uma falha no teste devido a uma configuração interna do teste.

d) Errada. Um erro na string de comparação em `Assertions.assertEquals` causaria uma falha no teste, mas a alternativa ‘b’ descreve uma falha mais fundamental relacionada à lógica do mock e do cenário. Além disso, a correção sugerida (“Falha ao processar pedido”) implicaria que o teste está esperando uma falha, o que contradiz a ideia de um “caso de sucesso” implícito na alternativa ‘b’.

e) Errada. Se o método `buscarPorId` não foi corretamente mockado para retornar um valor válido, o teste falharia, mas a alternativa ‘b’ aponta para uma falha na lógica de validação do mock, que é um ponto crítico para o fluxo de sucesso/falha.

Letra b.

021. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

O ciclo de desenvolvimento em PHP é mais lento que em Java, devido à necessidade de compilar o código a cada alteração antes de executá-lo.



O ciclo de desenvolvimento em PHP é geralmente mais rápido que em Java para pequenas alterações, precisamente porque PHP é uma linguagem interpretada (ou compilada Just-In-Time em tempo de execução por engines como Zend Engine/PHP-FPM), e não requer uma etapa de compilação explícita a cada alteração de código-fonte antes da execução, como é o caso do Java (que compila para bytecode). Em PHP, você pode salvar uma alteração e recarregar a página para ver o resultado imediatamente. Em Java, uma alteração no código-

fonte geralmente exige recompilação e, muitas vezes, reinício do servidor de aplicação para que as mudanças sejam efetivadas.

Errado.

022. (UFV/UFV-MG/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre as interfaces em Java, é INCORRETO afirmar que:

- a) podem ser usadas na marcação de classes, produzindo a noção de relacionamento do tipo “é um”.
- b) permitem compartilhar métodos e constantes comuns entre classes não relacionadas hierarquicamente.
- c) podem conter constantes, assinaturas de métodos, métodos estáticos e ser instanciadas diretamente por meio do construtor.
- d) funcionam como contratos de implementação para um componente, permitindo a implementação de funcionalidades por times diferentes.



- a) Certa. Interfaces podem ser usadas como “interfaces de marcação” (marker interfaces), como Serializable ou Cloneable, que não contêm métodos, mas indicam uma capacidade ou característica para a JVM. Elas também estabelecem um relacionamento “é um” (por exemplo, um Carro é um Veículo se Carro implementa a interface Veículo ou herda de uma classe que a implementa).
- b) Certa. Interfaces são excelentes para definir um contrato comum (métodos) e compartilhar constantes entre classes que não possuem uma relação de herança direta, mas que precisam ter um comportamento específico em comum.
- c) Errada. Interfaces em Java podem conter constantes (implicitamente public static final), assinaturas de métodos (implicitamente public abstract até Java 8, e public a partir de Java 9 para métodos default e static), e métodos estáticos (a partir de Java 8). No entanto, interfaces não podem ser instanciadas diretamente por meio de um construtor. Elas definem um contrato que deve ser implementado por classes concretas.
- d) Certa. Esta é uma das principais funções das interfaces: definir um contrato (um conjunto de métodos que uma classe deve implementar) que garante que diferentes componentes ou módulos (desenvolvidos por times diferentes) possam interagir de forma consistente.

Letra c.

023. (IBFC/UBFC/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Um computador é uma máquina que processa dados. Ao escrevermos um programa, estamos, na verdade, realizando essa tarefa com os dados que o usuário entrega para a máquina. Por exemplo,

quando queremos que um programa realize o cálculo da média entre duas notas, é preciso que, de alguma forma, o programa “receba” essas duas notas. Assinale a alternativa que apresenta qual operador é comumente utilizado em linguagens baseadas em C, como a linguagem Java por exemplo, para sinalizar o recebimento de um valor para uma variável.

- a) =
- b) <=
- c) <
- d) :=:
- e) =<



- a) Certa. O operador = (sinal de igual) é o operador de atribuição em Java, C, C++, e muitas outras linguagens. Ele é usado para atribuir um valor ao lado direito a uma variável no lado esquerdo. Ex: `int nota = 10;`
- b) Errada. <= é o operador relacional “menor ou igual a”.
- c) Errada. < é o operador relacional “menor que”.
- d) Errada. :=: não é um operador padrão em Java ou linguagens baseadas em C. É usado em algumas outras linguagens ou contextos específicos.
- e) Errada. =< não é um operador padrão em Java ou linguagens baseadas em C.

Letra a.

024. (CESPE/CEBRASPE/EMBRAPA/ANALISTA/ÁREA GESTÃO DA INFORMAÇÃO/2025) Julgue o próximo item, a respeito de arquitetura de sistemas de computação, sistemas de informação e linguagens de programação.

Em Java, a palavra-chave `static` é utilizada para declarar variáveis locais dentro de um método, para que elas mantenham seu valor entre diferentes chamadas ao método.



Em Java, a palavra-chave `static` é usada para declarar membros (atributos ou métodos) que pertencem à classe em si, e não a instâncias individuais da classe. Variáveis `static` são compartilhadas por todas as instâncias da classe e mantêm seu valor durante a vida útil do programa. No entanto, não é possível declarar variáveis locais dentro de um método como `static`. Variáveis locais são sempre criadas e destruídas a cada chamada do método, e não persistem entre chamadas. A persistência de valor entre chamadas de método para uma variável que não é um atributo de instância ou de classe não é uma característica de `static` em variáveis locais.

Errado.

025. (CESPE/CEBRASPE/EMBRAPA/ANALISTA/ÁREA GESTÃO DA INFORMAÇÃO/2025) Julgue o seguinte item, referente à programação e estruturas de dados.

O código a seguir, em Java, retornará Tecnológica como resultado de sua execução.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> embrapa = new
        ArrayList<String>();
        embrapa.add("Ageitec");
        embrapa.add("Agência");
        embrapa.add("Embrapa");
        embrapa.add("de Informação");
        embrapa.add("Tecnológica");
        int a=1;
        for (int i = 0; i < embrapa.size()-2; ++i)
        {
            a = a+2;
        }
        System.out.println(embrapa.get(a-3));
    }
}
```



Vamos analisar a execução do código:

Inicialização do ArrayList embrapa:

```
embrapa.add("Ageitec"); // índice 0
embrapa.add("Agência"); // índice 1
embrapa.add("Embrapa"); // índice 2
embrapa.add("de Informação"); // índice 3
embrapa.add("Tecnológica"); // índice 4
embrapa.size() é 5.
```

Inicialização de a:

```
int a = 1;
```

Loop for:

A condição do loop é $i < \text{embrapa.size()} - 2$, ou seja, $i < 5 - 2$, que é $i < 3$.

Iteração 1: $i = 0$. $a = a + 2 \rightarrow a = 1 + 2 = 3$.

Iteração 2: $i = 1$. $a = a + 2 \rightarrow a = 3 + 2 = 5$.

Iteração 3: $i = 2$. $a = a + 2 \rightarrow a = 5 + 2 = 7$.

Fim do loop: i se torna 3, a condição $i < 3$ é falsa.

Valor final de a : a é 7.

Instrução `System.out.println(embrapa.get(a - 3));`:

$a - 3$ é $7 - 3 = 4$.

`embrapa.get(4)` acessa o elemento no índice 4.

O elemento no índice 4 é "Tecnológica".

Conclusão: o código realmente retornará "Tecnológica". Portanto, a afirmação de que o código retornará "Tecnológica" está Correta.

Certo.

026. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

A compilação de código em linguagens como Java resulta em bytecode que é interpretado pela JVM, enquanto o PHP é interpretado diretamente sem a necessidade de uma fase de compilação, o que pode levar a um desempenho menor, mas, por outro lado, teremos mais facilidades de depuração.



A primeira parte da afirmação sobre Java está correta: o código Java é compilado para bytecode, que é então executado pela JVM. No entanto, a afirmação sobre PHP está desatualizada ou imprecisa. Embora PHP seja tradicionalmente vista como uma linguagem interpretada, as versões modernas do PHP (a partir do PHP 5.5 com o Zend OpCache e especialmente PHP 7+) utilizam uma fase de compilação para opcode (um tipo de bytecode PHP) que é então executado. Esse opcode pode ser armazenado em cache para evitar recompilações repetidas. Portanto, PHP não é "interpretado diretamente sem a necessidade de uma fase de compilação". Além disso, a facilidade de depuração não é necessariamente maior em PHP por essa razão; ambas as linguagens possuem ferramentas de depuração robustas.

Errado.

027. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

Em Java, os tipos de dados primitivos são passados por referência, permitindo que suas mudanças dentro de um método afetem diretamente a variável original.



Em Java, os tipos de dados primitivos (int, char, boolean, double, float, long, short, byte) são passados por valor para os métodos. Isso significa que uma cópia do valor da variável é passada para o método, e qualquer alteração feita nessa cópia dentro do método não afeta a variável original fora do método. Apenas objetos (tipos de referência) são passados por referência (na verdade, a referência ao objeto é passada por valor, mas isso permite que o objeto original seja modificado).

Errado.

028. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) Em relação às linguagens Java e PHP, julgue o item a seguir.

A administração de memória em Java é automatizada pelo sistema de garbage collection, que identifica e libera a memória ocupada por objetos que não são mais referenciados, evitando-se vazamentos de memória.



Uma das características mais importantes do Java é o gerenciamento automático de memória através do Garbage Collector (Coletor de Lixo). Ele monitora os objetos na memória e, quando detecta que um objeto não está mais sendo referenciado por nenhuma parte ativa do programa, ele o marca para remoção e libera a memória que ele ocupava. Isso ajuda a prevenir vazamentos de memória e simplifica o desenvolvimento, pois o programador não precisa gerenciar explicitamente a alocação e desalocação de memória.

Certo.

029. (FGV/PC-MG/PERITO CRIMINAL/2025) Considere as características fundamentais da linguagem Java e seus recursos no desenvolvimento de aplicações. Com base nos conceitos básicos de orientação a objetos, em relação ao uso de classes e objetos no Java, assinale a afirmativa CORRETA:

- a) Em Java, uma classe pode ser instanciada apenas se tiver um construtor público ou um construtor com especificadores de acesso adequados.
- b) O modificador public impede que um método ou variável de instância seja acessado fora da sua própria classe.

- c) Uma classe semiabstrata não pode ser instanciada diretamente, mas pode conter métodos implementados e não implementados.
- d) O conceito de polimorfismo permite que um objeto se comporte de forma diferente dependendo do tipo da referência usada para acessá-lo.
- e) O uso da palavra-chave `super` é utilizado para acessar membros da classe pai em classes que herdam de outra classe.



- a) Errada. Uma classe pode ser instanciada se tiver um construtor acessível (público, protegido, ou padrão/package-private dependendo do contexto de acesso). No entanto, a afirmação “apenas se tiver um construtor público” é muito restritiva. Uma classe pode ter um construtor private (como no padrão Singleton) e ainda ser instanciada internamente ou através de um método estático.
- b) Errada. O modificador `public` faz exatamente o oposto: ele permite que um método ou variável de instância seja acessado de qualquer lugar, ou seja, fora da sua própria classe, por outras classes, pacotes, etc.
- c) Errada. O termo “classe semiabstrata” não é um conceito padrão em Java. Classes são ou concretas (podem ser instanciadas) ou abstratas (não podem ser instanciadas diretamente). Classes abstratas podem ter métodos implementados e não implementados.
- d) Errada. O polimorfismo em Java permite que um objeto de uma subclasse seja tratado como um objeto de sua superclasse ou de uma interface que ele implementa. O comportamento do método chamado (se sobrescrito) é determinado pelo tipo real do objeto em tempo de execução (ligação dinâmica), não pelo tipo da referência usada para acessá-lo. Por exemplo, se `Animal a = new Cachorro();`, e `Cachorro` sobrescreve `fazerSom()`, `a.fazerSom()` chamará o método de `Cachorro`, mesmo que a referência seja `Animal`.
- e) Certa. A palavra-chave `super` é usada em uma subclasse para se referir a membros (atributos e métodos) da sua superclasse imediata. É comumente usada para chamar o construtor da superclasse (`super(...)`) ou para invocar um método sobrescrito da superclasse (`super.metodo()`).

Letra e.

030. (FCC/PREFEITURA DE SÃO PAULO–SP/ANALISTA DE PLANEJAMENTO E DESENVOLVIMENTO ORGANIZACIONAL TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO/2025) Uma prefeitura está desenvolvendo um sistema integrado de gestão pública que centraliza diversas funções administrativas. Para garantir a consistência dos *logs* de sistema, foi decidido implementar o padrão *Singleton* para a classe de gerenciamento de logs. Um dos desenvolvedores sugeriu a seguinte implementação em Java:

```
public class LogManager {
    private static LogManager instance;
    private LogManager() {}

    public static LogManager getInstance() {
        if (instance == null) {
            instance = new LogManager();
        }
        return instance;
    }
}
```

Em condições ideais, para garantir que apenas um thread possa executar o método `getInstance` por vez, evitando a criação de múltiplas instâncias em um ambiente *multithread*, e considerando que o desempenho não é a principal prioridade, é adequado:

- a) adicionar o modificador *synchronized* ao método *getInstance* para torná-lo *thread-safe*.
- b) declarar o atributo *instance* como *flexible* para garantir a consistência em ambientes *multithread*.
- c) implementar a inicialização precoce (*lazy initialization*), instanciando *instance* imediatamente após a declaração.
- d) redefinir o construtor como público, garantindo assim que apenas uma instância seja criada a partir de chamadas externas.
- e) adicionar um bloco *try-catch* ao método *getInstance* para lidar com a exceção *MultiThreadException*.



- a) Certa. Adicionar o modificador *synchronized* ao método `getInstance()` é a forma mais simples e direta de garantir que apenas um thread por vez possa executar o método. Isso previne a condição de corrida que poderia levar à criação de múltiplas instâncias do Singleton em um ambiente *multithread*. Embora possa ter um impacto no desempenho (pois serializa o acesso), a questão explicitamente afirma que “o desempenho não é a principal prioridade”.
- b) Errada. “flexible” não é um modificador de acesso ou palavra-chave válida em Java para atributos. O modificador *volatile* é usado para garantir visibilidade de escritas em memória entre threads, mas não garante atomicidade na criação da instância por si só.
- c) Errada. A inicialização precoce (*eager initialization*), onde a instância é criada no momento da declaração do atributo estático (`private static Singleton instance = new Singleton();`), é uma forma de garantir *thread-safety* sem *synchronized* no método `getInstance()`. No entanto, a questão fala em “adicionar o modificador *synchronized* ao método *getInstance*”, o que sugere que a implementação inicial é de inicialização preguiçosa (*lazy initialization*) e

precisa de sincronização. Além disso, a inicialização precoce não é uma “adição” ao método `getInstance()`.

d) Errada. Redefinir o construtor como public violaria o princípio fundamental do padrão Singleton, que é ter um construtor privado para impedir a criação de instâncias externas. Isso permitiria a criação de múltiplas instâncias, o oposto do objetivo.

e) Errada. Não existe uma `MultithreadException` padrão em Java que precise ser tratada dessa forma para garantir a thread-safety de um Singleton. O tratamento de exceções não é o mecanismo para garantir a atomicidade na criação da instância.

Letra a.

031. (FUNDATEC/IF-AM/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Sobre a declaração ‘import’ na linguagem de programação Java, como, por exemplo “import java.util.Scanner;” analise as assertivas abaixo:

I – É uma declaração que auxilia o compilador a localizar as classes utilizadas no programa.

II – Indica que o programa faz uso de uma classe pré-definida do pacote Java.

III – O compilador não garante que o uso da classe será sempre correto e adequado ao contexto, não sendo necessário nenhum teste adicional.

Quais estão corretas?

a) Apenas I.

b) Apenas III.

c) Apenas I e II.

d) Apenas II e III.

e) I, II e III.



I – Certa. A declaração `import` informa ao compilador onde encontrar as classes que estão sendo referenciadas no código, sem a necessidade de usar o nome totalmente qualificado (pacote.classe) a cada vez. Por exemplo, `import java.util.Scanner;` permite que você use `Scanner` em vez de `java.util.Scanner`.

II – Certa. `java.util.Scanner` é de fato uma classe pré-definida (parte da API padrão do Java) que reside no pacote `java.util`. A declaração `import` é usada para trazer classes de outros pacotes para o escopo do arquivo atual.

III – Errada. O compilador Java realiza uma forte verificação de tipos e sintaxe. Ele garante que o uso da classe esteja sintaticamente correto e que os tipos de dados sejam compatíveis. Por exemplo, se você tentar chamar um método que não existe na classe, o compilador gerará um erro. No entanto, o compilador não pode garantir a lógica de negócio ou a adequação ao contexto em tempo de execução (por exemplo, se um `Scanner` tentará ler um arquivo que

não existe, isso é um erro de execução, não de compilação). A segunda parte da frase “não sendo necessário nenhum teste adicional” é definitivamente falsa; testes (unitários, de integração, etc.) são sempre necessários para garantir a correção e adequação do software. Portanto, apenas as assertivas I e II estão corretas.

Letra c.

032. (CESGRANRIO/UNEMAT/ANALISTA DE SISTEMAS/2024) No processo de geração de código executável em Java, a etapa crucial que envolve a transformação do código-fonte Java em bytecode e o componente responsável por executar esse bytecode são, respectivamente:

- a) bytecode e JIT Compiler
- b) compilação e JRE
- c) compilação e JVM
- d) interpretação e JRE
- e) intermediário e Bytecode Engine



- a) Errada. Bytecode é o resultado da compilação, não a etapa. Já o JIT Compiler (Just-In-Time) é parte da execução, mas atua dentro da JVM, e não é o componente principal responsável pela execução.
- b) Errada. A etapa está correta (compilação gera bytecode), mas o JRE (Java Runtime Environment) é um conjunto de ferramentas, não o executor direto. Quem executa o bytecode é a JVM, que faz parte do JRE.
- c) Certa. Compilação é a etapa que converte o código-fonte.java em bytecode.class, e a JVM (Java Virtual Machine) é a responsável por executar o bytecode.
- d) Errada. Embora a JVM possa interpretar bytecode, o termo “interpretação” não descreve a etapa de transformação do código-fonte em bytecode. Além disso, como dito, o JRE é um ambiente, não o executor direto.
- e) Errada. “Intermediário” não é o nome de uma etapa formal do processo. E Bytecode Engine não é um componente padrão da terminologia Java — o executor correto é a JVM.

Letra c.

033. (UEM/UEM/ANALISTA DE INFORMÁTICA I/2025) Qual das alternativas apresenta uma instrução válida em Java padrão, sem necessidade de bibliotecas externas, e que imprime uma mensagem no console?

- a) `print("Olá, mundo!")`
- b) `printf("Olá, mundo!");`

- c) `echo "Olá, mundo!";`
- d) `System.out.println("Olá, mundo!");`
- e) `console.log("Olá, mundo!");`



- a) Errada. O método `print()` existe em Java, mas precisa ser chamado a partir de um objeto ou classe, como `System.out.print()`.
- b) Errada. Assim como o `print()`, o método `printf()` precisa ser chamado como `System.out.printf()` em Java. Sem o prefixo, causa erro de compilação.
- c) Errada. `echo` não é um comando de linguagem Java. É usado em linguagens PHP e em shells Unix.
- d) Certa. Essa é a forma correta e padrão para imprimir uma mensagem no console. Utiliza o método `println()` da saída padrão `System.out`.
- e) Errada. `console.log()` é usado em JavaScript, não em Java.

Letra d.

034. (QUADRIX/CFP/TÉCNICO ADMINISTRATIVO/SUPORTE TÉCNICO EM INFORMÁTICA/2024)

Quanto às linguagens de programação Java e PHP, julgue o item a seguir.

O laço de repetição `do-while`, na linguagem Java, avalia primeiro a expressão condicional e, em seguida, executa o comando.



Para compreender a questão proposta, é fundamental entender o funcionamento do laço de repetição `do-while` na linguagem de programação Java. A questão trata de um conceito básico relacionado às estruturas de controle de fluxo, que são elementos essenciais em qualquer linguagem de programação.

O laço `do-while` possui uma característica distinta: ele sempre executa o bloco de comandos ao menos uma vez, independentemente do resultado da condição, pois a verificação ocorre somente após a execução.

No enunciado da questão, afirma-se que o `do-while` “avalia primeiro a expressão condicional e, em seguida, executa o comando”. Essa afirmação está incorreta, uma vez que, na estrutura `do-while`, a execução do bloco de código ocorre antes da avaliação da condição. Portanto, a ordem correta é: executar primeiro, avaliar depois.

Errado.

035. (INSTITUTO CONSULPLAN/HEMOBRÁS/ANALISTA INDUSTRIAL DE HEMODERIVADOS E BIOTECNOLOGIA/TECNOLOGIA DA INFORMAÇÃO E OPERAÇÃO/2025) Estruturas de controle

de fluxo são fundamentais em *Java* porque permitem que o programador controle o comportamento e a lógica do programa. Em *Java*, qual é a saída do seguinte código que utiliza um *loop* aninhado?

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.print(i * j + " ");
            }
            System.out.println();
        }
    }
}
```

- a) 1 4 9.
- b) 1 2 4 6.
- c) 1 2 2 4 3 6.
- d) Nenhuma saída.



1 – Loop:

1*1 = 1

1*2 = 2

2 – Loop:

2*1 = 2

2*2 = 4

3 – Loop:

3*1 = 3

3*2 = 6

//1 2 2 4 3 6

Letra c.

036. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Em Java, a recursão é uma técnica comum, na qual uma função pode se chamar repetidamente até atingir uma condição de parada, sendo utilizada em problemas como pesquisa binária e algoritmos de divisão e conquista.



Recursão é uma técnica de programação onde uma função (ou método, em Java) chama a si mesma para resolver um problema. Para evitar um loop infinito, deve haver uma “condição de parada” (caso base) que encerra as chamadas recursivas. É uma técnica poderosa e comum para resolver problemas que podem ser divididos em subproblemas menores do mesmo tipo, como na pesquisa binária, cálculo de fatorial, travessia de árvores, e algoritmos de divisão e conquista (como Merge Sort ou Quick Sort).

Certo.

037. (IMPARH/CGM DE FORTALEZA–CE/AUDITOR DE CONTROLE INTERNO/2025) Sobre os conceitos relacionados à orientação a objetos na linguagem de programação Java, marque a opção CORRETA.

- a) Java permite herança múltipla de classes diretamente.
- b) Todos os métodos de uma classe abstrata devem ser abstratos.
- c) Classes em Java podem implementar várias interfaces.
- d) Uma classe com métodos e variáveis públicas não pode ser encapsulada.



- a) Errada. Java não permite herança múltipla de classes diretamente. Uma classe só pode estender uma única superclasse. Herança múltipla só é permitida via interfaces.
- b) Errada. Uma classe abstrata pode conter métodos abstratos e métodos concretos (com corpo). Não é obrigatório que todos os métodos sejam abstratos.
- c) Certa. Java permite que uma classe implemente múltiplas interfaces, o que é uma forma de permitir herança múltipla de comportamento, sem os conflitos da herança múltipla de classes.
- d) Errada. Encapsulamento é o princípio de ocultar os detalhes internos de uma classe e controlar o acesso a seus membros. Ainda que todos os membros sejam públicos, a classe pode ser encapsulada, embora isso vá contra as boas práticas.

Letra c.

038. (CESPE/CEBRASPE/TRT 10ª REGIÃO/ANALISTA JUDICIÁRIO/ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Acerca de desenvolvimento de sistemas, julgue o próximo item:
Em Java, uma das maneiras de se implementar o polimorfismo é a partir de interfaces que permitem o tratamento de objetos de classes diferentes de forma uniforme.



O polimorfismo em Java pode ser alcançado de várias maneiras, e uma das mais poderosas é através de interfaces. Quando diferentes classes implementam a mesma interface, elas concordam em fornecer implementações para os métodos definidos nessa interface. Isso permite que objetos dessas classes diferentes sejam referenciados pelo tipo da interface e tratados de forma uniforme, independentemente de suas classes concretas. Por exemplo, se Carro e Moto implementam a interface Veiculo, você pode ter uma lista de Veiculos e chamar o método ligar() em cada um, sem se preocupar se é um Carro ou uma Moto.

Certo.

039. (IBAM/PRODESAN-SP/ANALISTA DE SUPORTE/2025) Os comandos de entrada e saída permitem a interação entre o programa e o usuário, sendo responsáveis pela leitura de dados e exibição de resultados. Acerca desse tema, marque (V), para as afirmativas verdadeiras, e (F), para as falsas.

Em linguagens como C, a função scanf permite a leitura de múltiplos valores de entrada, desde que sejam especificados os tipos correspondentes no formato de entrada.

A função print em Python é utilizada para exibir dados na saída padrão, sendo possível utilizar múltiplos argumentos separados por vírgulas para concatenar textos e valores.

A função cin em C++ lê dados de entrada diretamente do teclado, exigindo que o tipo da variável seja especificado na chamada da função.

Em linguagens como Java, o método System.out.println é utilizado para exibir dados na saída padrão, adicionando automaticamente uma quebra de linha ao final da impressão.

A sequência está correta em:

- a) V – V – V – V.
- b) V – V – F – V.
- c) V – F – F – F.
- d) F – F – V – V.



(V) Em linguagens como C, a função scanf permite a leitura de múltiplos valores de entrada, desde que sejam especificados os tipos correspondentes no formato de entrada.

A função scanf em C é muito flexível e permite ler múltiplos valores de diferentes tipos em uma única chamada, usando especificadores de formato como %d para inteiros, %f para floats, %s para strings, etc. Ex: scanf("%d %f", &idade, &altura);

(V) A função print em Python é utilizada para exibir dados na saída padrão, sendo possível utilizar múltiplos argumentos separados por vírgulas para concatenar textos e valores.

A função `print()` em Python aceita múltiplos argumentos separados por vírgulas, que são impressos na saída padrão, separados por um espaço por padrão. Ex: `nome = "Ana"; idade = 30; print("Nome:", nome, "Idade:", idade);`.

(F) A função `cin` em C++ lê dados de entrada diretamente do teclado, exigindo que o tipo da variável seja especificado na chamada da função.

A função `cin` em C++ lê dados de entrada do teclado. No entanto, ela é `type-safe` e não exige que o tipo da variável seja especificado na chamada da função. O tipo é inferido automaticamente pela variável para a qual o valor está sendo lido. Ex: `int numero; cin >> numero;` (o tipo `int` já está na declaração da variável, não na chamada de `cin`).

(V) Em linguagens como Java, o método `System.out.println` é utilizado para exibir dados na saída padrão, adicionando automaticamente uma quebra de linha ao final da impressão. O método `System.out.println()` em Java imprime o argumento fornecido na saída padrão e, em seguida, avança para a próxima linha (adiciona uma quebra de linha). Existe também `System.out.print()` que não adiciona a quebra de linha.

A sequência correta é V – V – F – V.

Letra b.

040. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

No Java, os subprogramas são definidos por meio de métodos, que podem ser classificados como funções (que retornam um valor) ou procedimentos (que não retornam valor), sendo ambos componentes essenciais para a modularização do código.



Em Java, os blocos de código que realizam tarefas específicas são chamados de métodos. Esses métodos podem ser categorizados de forma análoga a funções e procedimentos de outras linguagens:

- Funções: métodos que retornam um valor (possuem um tipo de retorno diferente de `void`);
- Procedimentos: métodos que não retornam um valor (possuem o tipo de retorno `void`). Ambos são cruciais para a modularização do código, permitindo que programas complexos sejam divididos em partes menores, mais gerenciáveis e reutilizáveis.

Certo.

041. (FUNDATEC/PREFEITURA DE TANGARÁ DA SERRA/ANALISTA DE SISTEMAS/2025) Analise o trecho de código abaixo, escrito na linguagem de programação Java, e assinale a alternativa que corresponde a uma forma adequada de criar um objeto a partir da classe Filme.

```
class Filme {
    String titulo;

    String getTitulo() {
        return titulo;
    }

    void setTitulo(String titulo) {
        this.titulo = titulo;
    }
}
```

- a) new f = Filme();
- b) Filme f = new Filme();
- c) (Filme) f = Filme;
- d) f = new Filme;



- a) Errada. new é usado para invocar o construtor, mas a declaração da variável deve vir antes.
- b) Certa. Esta é a sintaxe padrão e correta para criar uma nova instância de uma classe em Java. Filme f declara uma variável f do tipo Filme, e new Filme() cria um novo objeto da classe Filme e o atribui à variável f.
- c) Errada. (Filme) f seria um cast, mas Filme sozinho não é uma instância ou uma referência válida para atribuição.
- d) Errada. Está quase correta, mas falta o parêntese () após Filme para invocar o construtor (mesmo que seja o construtor padrão sem argumentos). Em Java, é new Filme().

Letra b.

042. (IDEGS/PREFEITURA DE CARIACICA-ES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SISTEMA DA INFORMAÇÃO/2025) Na linguagem de programação Java, pode-se estabelecer contratos que definem um conjunto de métodos (somente assinaturas dos métodos) que uma classe deve implementar. Este contrato é realizado para especificar um comportamento que várias classes podem compartilhar, sem forçar uma relação de herança direta entre elas. O trecho refere-se à(ao):

- a) Interface.
- b) Encapsulamento.
- c) Método abstrato.
- d) Classe Abstrata.



- a) Certa. A descrição se encaixa perfeitamente no conceito de Interface em Java. Interfaces definem um contrato (um conjunto de métodos abstratos que devem ser implementados por qualquer classe que a implemente), permitindo polimorfismo e compartilhamento de comportamento sem a necessidade de herança de implementação.
- b) Errada. Encapsulamento é o princípio de ocultar os detalhes internos de um objeto e expor apenas uma interface controlada para interagir com ele. Não se refere a contratos de métodos.
- c) Errada. Um método abstrato é um método sem implementação, que deve ser implementado por subclasses. Embora interfaces contenham métodos abstratos (implicitamente), a descrição se refere ao conceito mais amplo de “contrato” que é a interface em si.
- d) Errada. Uma classe abstrata pode conter métodos abstratos e métodos concretos, e não pode ser instanciada. Ela é usada para fornecer uma base comum para subclasses, mas ainda estabelece uma relação de herança (extends), o que a descrição explicitamente diz que não é forçado (“sem forçar uma relação de herança direta”).

Letra a.

043. (IDESG/PREFEITURA DE CARIACICA-ES/ANALISTA EM TECNOLOGIA DA INFORMAÇÃO/SISTEMA DA INFORMAÇÃO/2025) Considerando a linguagem de programação Java, este mecanismo consiste em um tipo especial de classe que define um conjunto fixo de constantes (ou valores imutáveis). Ele é usado quando se tem a intenção de representar um grupo de valores relacionados e constantes de forma mais legível e segura no código, viabilizando uma leitura de código mais prática e limpa. Desta forma, qual mecanismo da linguagem de programação Java está sendo descrito?

- a) Atributo privado.
- b) Enum.
- c) Modificadores de acesso.
- d) Constante.



- a) Errada. Um atributo privado é uma variável de instância que só pode ser acessada dentro da própria classe. Não define um conjunto fixo de constantes relacionadas.

b) Certa. A descrição se refere a um enum (enumeração) em Java. Um enum é um tipo de dado especial que permite definir um conjunto fixo de constantes nomeadas. É ideal para representar coleções de valores relacionados, como dias da semana, meses, status de um pedido etc., tornando o código mais legível e seguro (type-safe).

c) Errada. Modificadores de acesso (public, private, protected, default) controlam a visibilidade de classes, métodos e atributos. Não são um mecanismo para definir um conjunto de constantes.

d) Errada. Uma constante (declarada com final em Java) é um valor que não pode ser alterado após a inicialização. Embora um enum contenha constantes, a descrição se refere ao mecanismo que agrupa essas constantes de forma estruturada e type-safe, que é o enum em si, não apenas uma constante individual.

Letra b.

044. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Java não suporta tipos de dados primitivos, sendo obrigado a utilizar apenas objetos para representar valores como inteiros, caracteres e booleanos.



Java suporta sim tipos de dados primitivos. Na verdade, Java possui um conjunto de 8 tipos primitivos (byte, short, int, long, float, double, char, boolean) que são usados para armazenar valores simples diretamente, sem a sobrecarga de um objeto. Embora Java seja fortemente orientada a objetos e forneça classes wrapper (como Integer, Character, Boolean) para representar esses tipos primitivos como objetos quando necessário (por exemplo, em coleções), o uso de tipos primitivos é fundamental e otimizado para desempenho.

Errado.

045. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Em Java, as variáveis de tipo primitivo não consomem memória, pois são apenas referências a objetos armazenados na heap.



Em Java:

- Variáveis de tipo primitivo (int, char, boolean etc.) consomem memória e armazenam seus valores diretamente na stack (pilha) de execução do método onde foram declaradas (ou na área de dados estáticos se forem static). Elas não são referências a objetos;
- Variáveis de tipo de referência (objetos) armazenam referências (endereços de memória) para objetos que, por sua vez, são armazenados na heap. A referência em si é armazenada na stack (ou área de dados estáticos), mas o objeto para o qual ela aponta está na heap. Portanto, variáveis primitivas não são referências a objetos e consomem memória para armazenar seus próprios valores.

Errado.

046. (QUADRIX/CFO/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO/2025) As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

O Java oferece uma forte tipagem estática, o que significa que os tipos de dados de variáveis devem ser explicitamente definidos em tempo de compilação, garantindo maior segurança e detecção precoce de erros.



Java é uma linguagem de tipagem forte e estática.

- Tipagem forte: significa que o Java impõe regras rígidas sobre a compatibilidade de tipos, minimizando conversões implícitas que poderiam levar a erros;
- Tipagem estática: significa que os tipos das variáveis são verificados em tempo de compilação (antes da execução do programa) e devem ser explicitamente declarados. Isso permite que muitos erros relacionados a tipos sejam detectados cedo no ciclo de desenvolvimento, aumentando a segurança e a robustez do código.

Certo.

047. (AVANÇA SP/UNITAU/PROGRAMADOR PLENO/2025) Em Java, qual palavra-chave é usada para definir uma classe que não pode ser estendida?

- a) static
- b) final
- c) private
- d) protected
- e) abstract



- a) Errada. `static` é usado para membros de classe (atributos e métodos) que pertencem à classe em si, não a instâncias. Não impede a extensão de uma classe.
- b) Certa. A palavra-chave `final` quando aplicada a uma classe significa que essa classe não pode ser estendida (não pode ter subclasses). Isso é usado para garantir que o comportamento da classe não seja alterado por herança.
- c) Errada. `private` é um modificador de acesso que restringe a visibilidade de membros (atributos, métodos, construtores) à própria classe. Não pode ser aplicado a uma classe `top-level` para impedir sua extensão.
- d) Errada. `protected` é um modificador de acesso que permite que membros sejam acessados dentro da própria classe, por subclasses e por classes no mesmo pacote. Não impede a extensão de uma classe.
- e) Errada. `abstract` é usado para definir uma classe abstrata, que não pode ser instanciada diretamente e pode conter métodos abstratos. Classes abstratas são feitas para serem estendidas.

Letra b.

048. (UFRR/UFRR/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/2025) Python e Java são linguagens de programação muito populares e amplamente utilizadas. Embora ambas sejam linguagens de alto nível, ambas possuem diferenças e semelhanças em alguns aspectos. Analise as alternativas abaixo e encontre a INCORRETA:

- a) Java possui uma sintaxe mais verbosa, enquanto Python possui uma sintaxe mais simples e legível.
- b) Python é conhecida por ser uma linguagem com tipagem dinâmica, ao passo que Java é reconhecida por sua tipagem estática.
- c) Java possui Garbage collector, porém o Python não tem.
- d) Spring é um framework Java e Flask um framework Python.
- e) Java não permite herança múltipla, já o Python permite.



- a) Certa. Java é conhecida por sua sintaxe mais formal e verbosa (ex: declaração explícita de tipos, chaves para blocos), enquanto Python é elogiada por sua sintaxe concisa e legível (ex: tipagem dinâmica, indentação para blocos).
- b) Certa. Java é uma linguagem de tipagem estática (os tipos das variáveis são verificados em tempo de compilação e não podem mudar), enquanto Python é de tipagem dinâmica (os tipos das variáveis são verificados em tempo de execução e podem mudar).
- c) Errada. Java possui um Garbage Collector (coletor de lixo) para gerenciamento automático de memória. Python também possui um Garbage Collector, que gerencia automaticamente

a memória, liberando objetos que não são mais referenciados. Portanto, a afirmação de que Python não tem é falsa.

d) Certa. Spring é um dos frameworks mais populares e amplamente utilizados para desenvolvimento de aplicações em Java. Flask é um microframework web popular para Python.

e) Certa. Java não permite herança múltipla de classes (uma classe só pode estender uma única superclasse), enquanto Python permite herança múltipla (uma classe pode herdar de múltiplas classes base).

Letra c.

049. (IF-ES/IF-ES/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) Uma escola deseja implementar um sistema para gerenciar suas listas de alunos e professores, utilizando um aplicativo Java. Como parte do processo de desenvolvimento, o Técnico de TI precisa compilar o código-fonte do aplicativo e garantir que ele seja executado em diferentes sistemas operacionais, sem necessidade de recompilação.

Qual das alternativas descreve CORRETAMENTE o motivo pelo qual os programas Java podem ser executados em várias plataformas?

a) O compilador Java converte o código-fonte em bytecodes, que são independentes de plataforma e executados pela Java Virtual Machine (JVM).

b) O código-fonte Java é diretamente interpretado pelo sistema operacional de cada plataforma.

c) O compilador Java converte o código-fonte em instruções específicas de hardware para cada sistema operacional.

d) A JVM compila o código-fonte Java em tempo de execução, gerando um arquivo executável para cada sistema operacional.

e) O ambiente de desenvolvimento integrado (IDE) utilizado para desenvolver o programa garante sua compatibilidade em todas as plataformas.



a) Certa. Este é o princípio fundamental da portabilidade do Java ("Write Once, Run Anywhere"). O código-fonte Java (.java) é compilado em bytecode (.class), que é um formato intermediário e independente de plataforma. Este bytecode é então executado por uma Java Virtual Machine (JVM) específica para cada sistema operacional.

b) Errada. O código-fonte Java não é diretamente interpretado pelo sistema operacional. Ele é compilado para bytecode e depois executado pela JVM.

c) Errada. O compilador Java não gera instruções específicas de hardware para cada sistema operacional. Essa é a função da JVM em tempo de execução (através do JIT compiler).

d) Errada. A JVM executa o bytecode. Embora a JVM contenha um compilador Just-In-Time (JIT) que pode compilar partes do bytecode para código de máquina nativo em tempo de execução para otimização, ela não gera um “arquivo executável” separado para cada sistema operacional a partir do código-fonte.

e) Errada. O IDE (Integrated Development Environment) é uma ferramenta de desenvolvimento que facilita a escrita, compilação e depuração do código. Ele não é o responsável pela compatibilidade multiplataforma do Java; essa compatibilidade é uma característica da linguagem e de sua arquitetura (JVM e bytecode).

Letra a.

050. (INÉDITA/2025) Qual das seguintes afirmações sobre métodos estáticos em Java é verdadeira?

- a) Podem acessar diretamente variáveis de instância.
- b) Podem ser chamados sem a criação de um objeto da classe.
- c) Devem ser declarados dentro de um método principal.
- d) Precisam de uma instância de classe para serem invocados.



Métodos estáticos pertencem à classe e não requerem uma instância para serem chamados, diferentemente dos métodos de instância, que operam no estado de um objeto específico.

Letra b.

REFERÊNCIAS

BAELDUNG. *Java Tutorials*. Disponível em: <https://www.baeldung.com/>.

Acesso em: 10 mai. 2025.

GEEKSFORGEEKS. *Java Tutorial*. Disponível em: <https://www.geeksforgeeks.org/java/>.

Acesso em: 11 mai. 2025.

ORACLE. *Java Documentation*. Disponível em: <https://docs.oracle.com/en/java/>.

Acesso em: 15 abr. 2025.

ORACLE. *Learn Java*. Disponível em: <https://dev.java/learn/>. Acesso em: 15 abr. 2025.

PROGRAMIZ. *Java Tutorial*. Disponível em: <https://www.programiz.com/java-programming/>. Acesso em: 28 mai. 2025.

QUINTÃO, P. L. *Notas de aula da disciplina "Tecnologia da Informação"*. 2025.

TUTORIALSPPOINT. *Java Tutorial*. Disponível em: <https://www.tutorialspoint.com/java/index.htm>. Acesso em: 24 abr. 2025.

W3SCHOOLS. *Java Tutorial*. Disponível em: <https://www.w3schools.com/java/>.

Acesso em: 28 mai. 2025.

Abra



caminhos



crie

futuros

gran.com.br

